



Status

Finished

Started

Monday, 16 June 2025, 10:49 AM

Completed

Monday, 16 June 2025, 11:00 AM

Duration

11 mins 7 secs

Grade

94.55 out of 130.00 (72.73%)

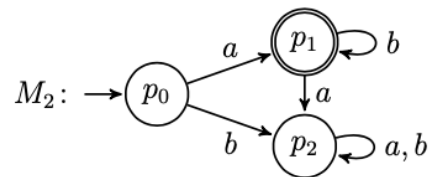
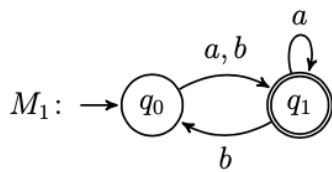
Question 1

Flag question

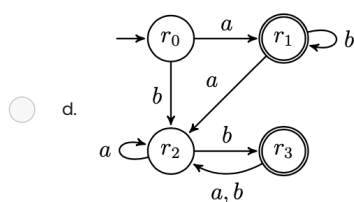
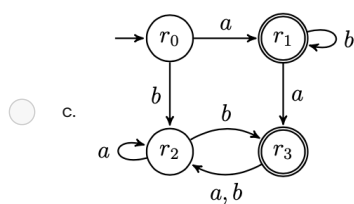
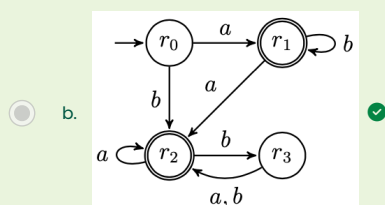
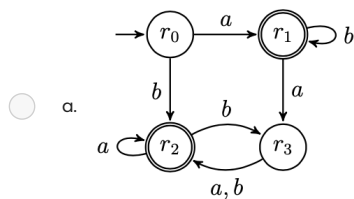
Correct

Mark 5.00 out of 5.00

Let M_1 and M_2 be the finite deterministic automata below, recognising the languages $L(M_1) = (a \cup b)(a \cup b(a \cup b))^*$ and $L(M_2) = ab^*$, respectively.

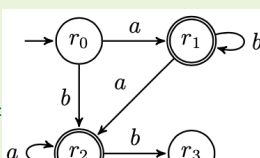


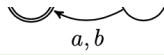
Which of the following automata recognises $L(M_1) \cup L(M_2)$ (the union of $L(M_1)$ and $L(M_2)$)?



😊 Your answer is correct.

The correct answer is:





Question 2

[Flag question](#)

Not answered

Marked out of 10.00

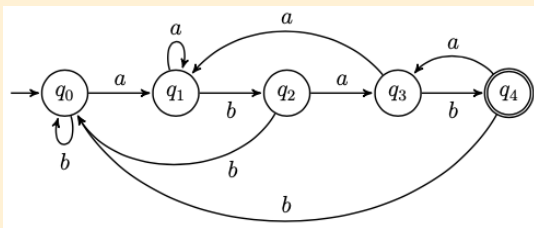
"Code-draw" the state diagram of a **deterministic** finite automaton recognising the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has suffix } abab\}$$

The syntax of reference to code-draw the automaton is that of [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

Solution:



NOTE: As the question specifically asked for a deterministic finite automaton, any solution using nondeterminism is considered incorrect.

```

#states
q0
q1
q2
q3
q4
#initial
q0
#accepting
q4
#alphabet
a
b
#transitions
q0:a>q1
q0:b>q0
q1:a>q1
q1:b>q2
q2:a>q3
q2:b>q0
q3:a>q1
q3:b>q4
q4:a>q3
q4:b>q0
  
```

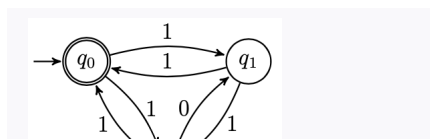
Question 3

Flag question

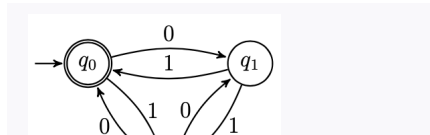
Not answered

Marked out of 5.00

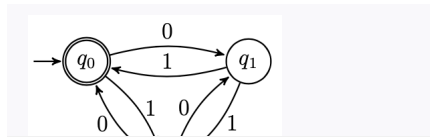
Drag each regular expression next to the corresponding equivalent automaton.



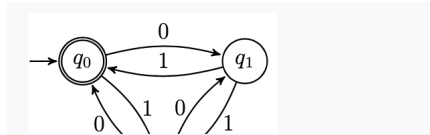
Drag answer here



Drag answer here



Drag answer here

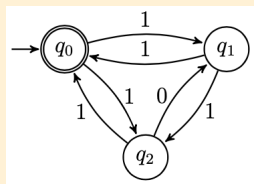
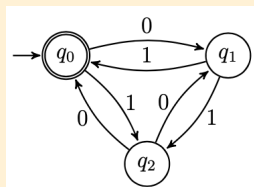
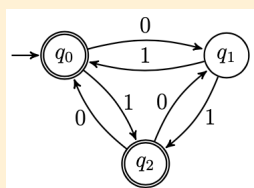
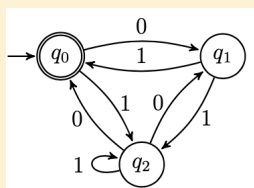


Drag answer here

 $(01 \cup (1 \cup 01)(01)^*(0 \cup 01))^*(\epsilon \cup (1 \cup 01)($
 $(11 \cup (1 \cup 11)(01)^*(1 \cup 01))^*$
 $(01 \cup (1 \cup 01)(1 \cup 01)^*(0 \cup 01))^*$
 $(01 \cup (1 \cup 01)(01)^*(0 \cup 01))^*$

☹ Your answer is incorrect.

The correct answer is:


 $(11 \cup (1 \cup 11)(01)^*(1 \cup 01))^*$

 $(01 \cup (1 \cup 01)(01)^*(0 \cup 01))^*$

 $(01 \cup (1 \cup 01)(01)^*(0 \cup 01))^*(\epsilon \cup (1 \cup 01)(01)^*)$

 $(01 \cup (1 \cup 01)(1 \cup 01)^*(0 \cup 01))^*$

Question 4

Flag question

Not answered

Marked out of 15.00

By using the Pumping Lemma, prove that the language $L = \{a^n(ab)^m \mid n, m \geq 0 \text{ and } n < m\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking the $\equiv$ symbol to expand the editor console, and then $\times$ to select the equation editor).

 Solution:

To show that L is nonregular we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to choose $w = a^p(ab)^{p+1}$. (Note that $w \in L$ and $|w| = p + 2(p+1) \geq p$. So we are playing according to the rules of the game).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$, and $|xy| \leq p$
4. For any decomposition of w into xyz as above, we show that for $i = 2$, $xy^iz \notin L$.
Note that by the condition $|xy| \leq p$ we have that xy contains only a 's.
Thus, $xy^2z = a^{p+|y|}(ab)^{p+1} \notin L$, because from the condition $y \neq \varepsilon$ we have that $|y| \geq 1$, so $p + |y| \geq p + 1$.

Question 5

Flag question

Correct

Mark 5.00 out of 5.00

Determine whether the following languages are regular or nonregular.

Select the correct option in the dropdown menus below:

- $\{w \in \{a, b\}^* \mid \text{the number of occurrences of } a\text{'s in } w \text{ are twice as those of } b\text{'s}\}$ is nonregular 
- $\{w \in \{a, b\}^* \mid w \text{ has an even number of occurrences of } a\text{'s}\}$ is regular 
- $\{a^n b^n \mid n \geq 0\} \cap \{(ab)^n \mid n \geq 0\}$ is regular 
- $\{a^{2n} b^n \mid n \geq 0\}$ is nonregular 
- $\{a^n a^{n+1} \mid n \geq 0\}$ is regular 

 Solution:

- $\{w \in \{a, b\}^* \mid \text{the number of occurrences of } a\text{'s in } w \text{ are twice as those of } b\text{'s}\}$ is **nonregular**.
- $\{w \in \{a, b\}^* \mid w \text{ has an even number of occurrences of } a\text{'s}\}$ is **regular**
- $\{a^n b^n \mid n \geq 0\} \cap \{(ab)^n \mid n \geq 0\}$ is **regular**
- $\{a^{2n} b^n \mid n \geq 0\}$ is **nonregular**
- $\{a^n a^{n+1} \mid n \geq 0\}$ is **regular**

Question 6

[Flag question](#)

Correct

Mark 5.00 out of 5.00

Complete the context-free grammar below so that it generates the language $\{a^{2n}b^m \mid n, m \geq 0 \text{ and } n \geq m\}$.

$S \rightarrow$ ☒ ☒ S ☒ $|$ ☒ ☒ S $|$ ☒

 **Solution:**

$$S \rightarrow aaSb \mid aaS \mid \varepsilon$$

Question 7

Flag question

Not answered

Marked out of 5.00

Match each context-free grammar to the left with its generated language.

$$S \rightarrow aB \mid \varepsilon$$

$$B \rightarrow bSc$$

Drag answer here

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow bBc \mid bc$$

Drag answer here

$$S \rightarrow A \mid B$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow bBc \mid bc$$

Drag answer here

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \varepsilon$$

$$B \rightarrow bBc \mid \varepsilon$$

Drag answer here

$$S \rightarrow aB$$

$$B \rightarrow bSc \mid \varepsilon$$

Drag answer here

 $\{a^n b^{n+m} c^m \mid n, m > 0\}$ $\{(ab)^n c^n \mid n > 0\}$ $\{a^n b^{n+m} c^m \mid n, m \geq 0\}$ $\{a(ba)^n c^n \mid n \geq 0\}$ $\{a(ba)^n c^n \mid n > 0\}$ $\{a^n b^n \mid n > 0\} \cup \{b^m c^m \mid m > 0\}$ $\{(ab)^n c^n \mid n \geq 0\}$
 Your answer is incorrect.

The correct answer is:

$$S \rightarrow aB \mid \varepsilon$$

$$B \rightarrow bSc$$
 $\{(ab)^n c^n \mid n \geq 0\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow bBc \mid bc$$
 $\{a^n b^{n+m} c^m \mid n, m > 0\}$

$$S \rightarrow A \mid B$$

$$A \rightarrow aAb \mid ab$$

$$B \rightarrow bBc \mid bc$$
 $\{a^n b^n \mid n > 0\} \cup \{b^m c^m \mid m > 0\}$

$$S \rightarrow AB$$

$$A \rightarrow aAb \mid \varepsilon$$

$$B \rightarrow bBc \mid \varepsilon$$
 $\{a^n b^{n+m} c^m \mid n, m \geq 0\}$

$$S \rightarrow aB$$

$$B \rightarrow bSc \mid \varepsilon$$
 $\{a(ba)^n c^n \mid n \geq 0\}$

Question 8

Flag question

Correct

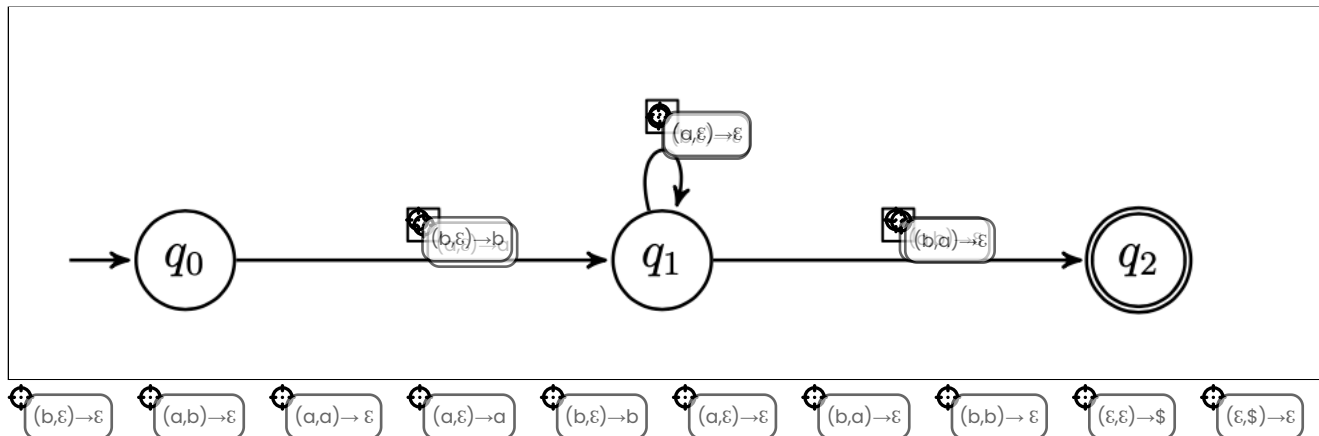
Mark 10.00 out of 10.00

Fill in the missing labels in the push-down automaton below so that it generates the language

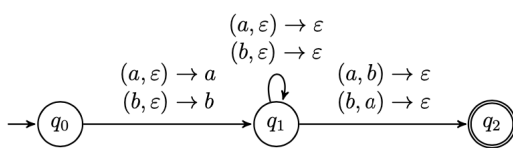
$L = \{w \mid w \in \{a, b\}^* \text{ starts and ends with different symbols}\}.$

Instructions:

- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



😊 Your answer is correct.

Solution:

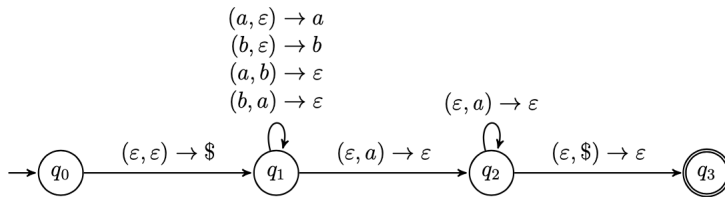
Question 9

Flag question

Correct

Mark 5.00 out of 5.00

Let M be the push-down automaton below:



Which of the following languages is recognised by M ?

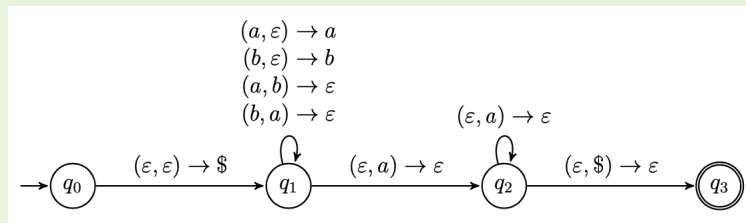
Select one:

- ☐ $\{w \mid w \in \{a, b\}^* \text{ has no more occurrences of } a\text{'s than } b\text{'s}\}$
- ☐ $\{w \mid w \in \{a, b\}^* \text{ has same or more occurrences of } a\text{'s than } b\text{'s}\}$
- ☒ $\{w \mid w \in \{a, b\}^* \text{ has strictly more occurrences of } a\text{'s than } b\text{'s}\}$ ✓
- ☐ $\{a^n b^m \mid n, m \geq 0 \text{ and } n \geq m\}$
- ☐ $\{b^n a^m \mid n, m \geq 0 \text{ and } n \geq m\}$

😊 Your answer is correct.

Solution:

The correct answer is $\{w \mid w \in \{a, b\}^* \text{ has strictly more occurrences of } a\text{'s than } b\text{'s}\}$.



The automaton works in the following way:

- The transition from q_0 to q_1 adds the symbol $\$$ to mark the bottom of the stack
- The self-loop in q_1 is responsible to match the occurrences of a 's and b 's. It does so by storing in the stack the input symbol just read or by matching/deleting the opposite symbol from the top of the stack.
Note that, if the input string has strictly more a 's than b 's there is always a way to nondeterministically choose the stack operations to perform so that we are left with a stack content of the form $aa^*\$$. Otherwise, the stack content won't be of the form $aa^*\$$.
- The transitions labelled by $(\varepsilon, a) \rightarrow \varepsilon$ are used to delete the remaining a symbols from the stack.
- If we reach the bottom of the stack (that we previously marked with $\$$), we can take the last transition to the accepting state q_3 . Note that this is possible only if the stack content after looping on q_1 is of the form $aa^*\$$.

The correct answer is: $\{w \mid w \in \{a, b\}^* \text{ has strictly more occurrences of } a\text{'s than } b\text{'s}\}$

Question 10

Flag question

Correct

Mark 5.00 out of 5.00

Consider the Bims expression $\text{res} = x + x \leq 2 \cdot y$.

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree for the evaluation of res according to the big-step operational semantics for typed Bims expressions, where we assume $s = [x \mapsto 2, y \mapsto 3]$.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

The proof tree diagram shows the evaluation of $\text{res} = x + x \leq 2 \cdot y$. The root node is $s \vdash \text{res} \rightarrow_E tt$. The tree branches into two main parts. The left part shows the evaluation of $x + x$ to 4, and the right part shows the evaluation of $2 \cdot y$ to 6. These are then combined using the leqBS rule to conclude $s \vdash \text{res} \rightarrow_E tt$. Various intermediate steps and rules are shown in boxes, some of which are already filled in.

😊 Your answer is correct.

Solution:

The solution proof tree shows the evaluation of $\text{res} = x + x \leq 2 \cdot y$. The tree is structured as follows:

- Top row: $s(x) = 2$, $s(x) = 2$, $\underline{2} \rightarrow_{\text{Int}} 2$, $s(y) = 3$
- Second row: $[\text{varBS}] \frac{s(x) = 2}{s \vdash x \rightarrow_E 2}$, $[\text{varBS}] \frac{s(x) = 2}{s \vdash x \rightarrow_E 2}$, $[\text{numBS}] \frac{\underline{2} \rightarrow_{\text{Int}} 2}{s \vdash \underline{2} \rightarrow_E 2}$, $[\text{varBS}] \frac{s(y) = 3}{s \vdash y \rightarrow_E 3}$
- Third row: $[\text{sumBS}] \frac{s \vdash x \rightarrow_E 2}{s \vdash x + x \rightarrow_E 4}$, $[\text{multBS}] \frac{s \vdash \underline{2} \rightarrow_E 2}{s \vdash 2 \cdot y \rightarrow_E 6}$
- Bottom row: $[\text{leqBS}] \frac{s \vdash x + x \rightarrow_E 4 \quad s \vdash 2 \cdot y \rightarrow_E 6}{s \vdash \text{res} \rightarrow_E tt}$

Question 11

Flag question

Correct

Mark 10.00 out of 10.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the *state model* big-step operational semantics for typed Bims statements, where

$W = \text{while } x \leq y \text{ do } B,$
 $B = x := x + \underline{1}; y := y - \underline{1},$

$s_0 = s[x \mapsto 1, y \mapsto 2],$
 $s_1 = s[x \mapsto 2, y \mapsto 2],$
 $s_2 = s[x \mapsto 2, y \mapsto 1].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$$\frac{}{s_0 \vdash x + \underline{1} \rightarrow_E 2}$$

[ass_{BS}]

$$\frac{}{s_1 \vdash y - \underline{1} \rightarrow}$$

[ass_{BS}]

$$\langle x := x + \underline{1}, s_0 \rangle \rightarrow s_1$$

[comp_{BS}]

$$\langle y := y - \underline{1}, s_1 \rangle \rightarrow$$

$$\frac{}{\langle W, s_0 \rangle \rightarrow s_2}$$

$$\langle B; W, s_0 \rangle \rightarrow s_2$$

$$\langle y := y - \underline{1}, \langle B, s_0 \rangle \rightarrow s_2 \rangle \rightarrow_E ff$$

$$\langle W, s_0 \rangle \rightarrow s_2$$

$$\langle W, s_2 \rangle \rightarrow s_2$$

$$\langle W, s_1 \rangle \rightarrow s_2$$

$$s_0 \vdash x + \underline{1} \rightarrow_E 2$$

$$s_2 \vdash x \leq y \rightarrow_E ff$$

$$s_1 \vdash y - \underline{1} \rightarrow_E 1$$

$$\langle x := x + \underline{1}, s_0 \rangle \rightarrow s_1$$

$$\langle y := y - \underline{1}, s_0 \rangle \rightarrow s_1$$

$$\langle B, s_0 \rangle \rightarrow s_2$$

[ass_{BS}]

[while-ff_{BS}]

[while-tt_{BS}]

[comp_{BS}]

$$\langle B; W, s_0 \rangle \rightarrow s_2$$

😊 Your answer is correct.

Solution:

$$\begin{array}{c}
 \frac{}{s_0 \vdash x + \underline{1} \rightarrow_E 2} \quad \frac{}{s_1 \vdash y - \underline{1} \rightarrow_E 1} \\
 \text{[ass}_{BS}\text{]} \quad \text{[ass}_{BS}\text{]} \\
 \frac{}{\langle x := x + \underline{1}, s_0 \rangle \rightarrow s_1} \quad \frac{}{\langle y := y - \underline{1}, s_1 \rangle \rightarrow s_2} \\
 \text{[comp}_{BS}\text{]} \quad \text{[while-ff}_{BS}\text{]} \\
 \frac{}{\langle B, s_0 \rangle \rightarrow s_2} \quad \frac{}{\langle W, s_2 \rangle \rightarrow s_2} \\
 \text{[comp}_{BS}\text{]} \\
 \frac{}{\langle B; W, s_0 \rangle \rightarrow s_2} \quad s_0 \vdash x \leq y \rightarrow_E tt \\
 \text{[while-tt}_{BS}\text{]} \\
 \frac{}{\langle W, s_0 \rangle \rightarrow s_2}
 \end{array}$$

Question 12

Flag question

Correct

Mark 10.00 out of 10.00

Let $W = \text{while } x \leq y \text{ do } x := x + 1; y := y - 1$. Fill in the blanks in the derivation sequence below according to the rules of the state model small-step semantics for typed Bims statements. (Expand the window of your browser for a better visualisation).

$\langle W, [x \mapsto 1, y \mapsto 6] \rangle$
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 1, y \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 1, y \mapsto 6] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 2, y \mapsto 6] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 2, y \mapsto 5] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 2, y \mapsto 5] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 2, y \mapsto 5] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 3, y \mapsto 5] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 3, y \mapsto 4] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 3, y \mapsto 4] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 3, y \mapsto 4] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 4, y \mapsto 4] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 3] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 4, y \mapsto 3] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 4, y \mapsto 3] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 4, y \mapsto 3]$ (skip_{SS})



$\langle W, [x \mapsto 1, y \mapsto 6] \rangle$
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 1, y \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 1, y \mapsto 6] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 2, y \mapsto 6] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 2, y \mapsto 5] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 2, y \mapsto 5] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 2, y \mapsto 5] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 3, y \mapsto 5] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 3, y \mapsto 4] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 3, y \mapsto 4] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 1; y := y - 1; W, [x \mapsto 3, y \mapsto 4] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 1; W, [x \mapsto 4, y \mapsto 4] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 3] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq y \text{ then } x := x + 1; y := y - 1; W \text{ else skip}, [x \mapsto 4, y \mapsto 3] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 4, y \mapsto 3] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 4, y \mapsto 3]$ (skip_{SS})

Question 13

Flag question

Partially correct

Mark 4.55 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the type system for typed Bims statements, where $E_1 = [x \mapsto \text{Int}, b \mapsto \text{Bool}]$.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

[num_{TS}]

$E_1(b) = \text{Bool}$

$E_1(x) = \text{Int}$

[var_{TS}]

$E_1 \vdash \text{if } b \text{ then } x := \underline{1} \text{ else skip: ok}$

[ass_{TS}]

$E_1 \vdash$

$E_1 \vdash b: \text{Bool}$

$x: \text{Int}$

$E_1 \vdash b: \text{Bool}$

$E_1 \vdash$

$E_1 \vdash x := \underline{1}: \text{Int}$

[if_{TS}]

$E_1 \vdash x: \text{ok}$

$E_1 \vdash \text{skip: ok}$

$E_1 \vdash x := \underline{1}: \text{ok}$

$E_1(b) = \text{Bool}$

[dec_{TS}]

[num_{TS}]

[ass_{TS}]

[if_{TS}]

[var_{TS}]

[skip_{TS}]

☹ Your answer is partially correct.
You have correctly selected 10.

Solution:

[num_{TS}]

$E_1(b) = \text{Bool}$

$E_1(x) = \text{Int}$

$E_1 \vdash 1: \text{Int}$

[var_{TS}]

$E_1 \vdash b: \text{Bool}$

[ass_{TS}]

$E_1 \vdash x := \underline{1}: \text{ok}$

[skip_{TS}]

$E_1 \vdash \text{skip: ok}$

$E_1 \vdash \text{if } b \text{ then } x := \underline{1} \text{ else skip: ok}$

Question 14

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation-tree constructed according to the rules of the type system for Bims statements with typed variable declarations. (Enlarge the window for a better view of the derivation).

$$\frac{\frac{\frac{x \notin \text{dom}(\emptyset)}{[\text{dec}_{\text{TS}}]} \quad \frac{\frac{\frac{\emptyset \vdash 3 : \text{Int}}{[\text{num}_{\text{TS}}]} \quad \frac{\frac{y \notin \text{dom}(E_1)}{[\text{dec}_{\text{TS}}]} \quad \frac{\frac{E_1 \vdash 2 : \text{Int}}{[\text{num}_{\text{TS}}]} \quad \frac{z \notin \text{dom}(E_2)}{[\text{dec}_{\text{TS}}]} \quad \frac{E_2 \vdash \text{var Int } z := 1; \text{skip} : \text{ok}}{[\text{skip}_{\text{TS}}]} \quad \frac{E_3 \vdash \text{skip} : \text{ok}}{[\text{skip}_{\text{TS}}]}}{E_2 \vdash \text{var Int } z := 1; \text{skip} : \text{ok}}}{E_1 \vdash \text{var Int } y := 2; \text{var Int } z := 1; \text{skip} : \text{ok}}}{\emptyset \vdash \text{var Int } x := 3; \text{var Int } y := 2; \text{var Int } z := 1; \text{skip} : \text{ok}}}$$

Provide the definitions of the typing environments E_1 , E_2 , and E_3 so that the above is a well-formed derivation proving that the statement `var Int x := 3; var Int y := 2; var Int z := 1; skip` is well-typed.

Select the appropriate values among the choices provided below:

$$\begin{array}{l} E_1 = [x \mapsto \text{Int} \quad \blacklozenge \quad \checkmark, y \mapsto \text{undefined} \quad \blacklozenge \quad \checkmark, z \mapsto \text{undefined} \quad \blacklozenge \quad \checkmark] \\ E_2 = [x \mapsto \text{Int} \quad \blacklozenge \quad \checkmark, y \mapsto \text{Int} \quad \blacklozenge \quad \checkmark, z \mapsto \text{undefined} \quad \blacklozenge \quad \checkmark] \\ E_3 = [x \mapsto \text{Int} \quad \blacklozenge \quad \checkmark, y \mapsto \text{Int} \quad \blacklozenge \quad \checkmark, z \mapsto \text{Int} \quad \blacklozenge \quad \checkmark] \end{array}$$

 **Solution:**

$$E_1 = [x \mapsto \text{Int}]$$
$$E_2 = [x \mapsto \text{Int}, y \mapsto \text{Int}]$$
$$E_3 = [x \mapsto \text{Int}, y \mapsto \text{Int}, z \mapsto \text{Int}]$$

Question 15

Flag question

Correct

Mark 5.00 out of 5.00

Choose the correct assertion about the following Bims statements and expressions:

- `var Int x := 5; var Bool y := 0; if y then x := x + 1 else x := x - 1` is

☐

ill-formed

☐

well-typed

☒

ill-typed



Mark 1.00 out of 1.00

The correct answer is: ill-typed

- `var Int x := 0; var Int y := 1` is

☒

ill-formed

☐

well-typed

☐

ill-typed

Mark 1.00 out of 1.00

The correct answer is: ill-formed

- `var Int y := 0; var Int x := x + 1; skip` is

☐

ill-formed

☐

well-typed

☒

ill-typed



Mark 1.00 out of 1.00

The correct answer is: ill-typed

- `var Bool x := 0 ≤ 0; var Int y := 0; skip` is

☐

ill-formed

☒

well-typed

☐

ill-typed

Mark 1.00 out of 1.00

The correct answer is: well-typed

- `var Bool x := true; var Int y := 0 ≤ x; skip` is

☐

ill-formed

☐

well-typed



ill-typed



Mark 1.00 out of 1.00

The correct answer is: ill-typed



Solution:

- `var Int x := 5; var Bool y := 0; if y then x := x + 1 else x := x - 1` is *ill-typed*, because the Boolean variable *y* is assigned an integer expression.
- `var Int x := 0; var Int y := 1` is *ill-formed*, because the second declaration does not specify the continuation, which is required by the formation rule.
- `var Int y := 0; var Int x := x + 1; skip` is *ill-typed*, because the assignment in the declaration of *x* uses an undeclared variable.
- `var Bool x := 0 ≤ 0; var Int y := 0; skip` is *well-typed*.
- `var Bool x := true; var Int y := 0 ≤ x; skip` is *ill-typed*, because the integer variable *y* is assigned a Boolean expression.

Question 16

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using fully-dynamic scope rules

$$\begin{array}{c}
 \sigma_3 \circ \mathcal{E}_3 \vdash x + y \rightarrow_E 9 \\
 \text{[assBS]} \frac{\sigma_2 \circ \mathcal{E}_2 \vdash x + \underline{1} \rightarrow_E 5 \quad \mathcal{E}_3, \pi_0 \vdash_{l_3} \langle y := x + y, \sigma_3 \rangle \rightarrow \sigma_4}{\sigma_2 \circ \mathcal{E}_2 \vdash x + \underline{1} \rightarrow_E 5} \\
 \text{[decBS]} \frac{\sigma_1 \circ \mathcal{E}_1 \vdash x + \underline{1} \rightarrow_E 4 \quad \mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{var Int } y := x + \underline{1}; y := x + y, \sigma_2 \rangle \rightarrow \sigma_4}{\sigma_1 \circ \mathcal{E}_1 \vdash x + \underline{1} \rightarrow_E 4} \\
 \text{[blockBS]} \frac{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var Int } x := x + \underline{1}; \text{var Int } y := x + \underline{1}; y := x + y, \sigma_1 \rangle \rightarrow \sigma_4}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{begin var Int } x := x + \underline{1}; \text{var Int } y := x + \underline{1}; y := x + y \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4} \\
 \text{[decBS]} \frac{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{3} \rightarrow_E 3 \quad \mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{begin var Int } x := x + \underline{1}; \text{var Int } y := x + \underline{1}; y := x + y \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4}{\mathcal{E}_0, \pi_0 \vdash_{l_0} \langle \text{var Int } x := \underline{3}; \text{begin var Int } x := x + \underline{1}; \text{var Int } y := x + \underline{1}; y := x + y \text{ end}, \sigma_0 \rangle \rightarrow \sigma_4}
 \end{array}$$

where $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the window of your browser for a better view of the derivation).

Provide the definitions of the stores and variable environments listed below by filling in the blanks so that the above is a well-formed derivation.

 $\sigma_0 = \emptyset$
 $\sigma_1 = [l_0 \mapsto 3, l_1 \mapsto \text{undefined}, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$
 $\sigma_2 = [l_0 \mapsto 3, l_1 \mapsto 4, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$
 $\sigma_3 = [l_0 \mapsto 3, l_1 \mapsto 4, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$
 $\sigma_4 = [l_0 \mapsto 3, l_1 \mapsto 4, l_2 \mapsto 9, l_3 \mapsto \text{undefined}]$
 $\mathcal{E}_1 = \mathcal{E}_0[x \mapsto l_0, y \mapsto \text{undefined}]$
 $\mathcal{E}_2 = \mathcal{E}_0[x \mapsto l_1, y \mapsto \text{undefined}]$
 $\mathcal{E}_3 = \mathcal{E}_0[x \mapsto l_1, y \mapsto l_2]$


Solution:

 $\sigma_0 = \emptyset$
 $\sigma_1 = [l_0 \mapsto 3]$
 $\sigma_2 = [l_0 \mapsto 3, l_1 \mapsto 4]$
 $\sigma_3 = [l_0 \mapsto 3, l_1 \mapsto 4, l_2 \mapsto 5]$
 $\sigma_4 = [l_0 \mapsto 3, l_1 \mapsto 4, l_2 \mapsto 9]$
 $\mathcal{E}_1 = \mathcal{E}_0[x \mapsto l_0]$
 $\mathcal{E}_2 = \mathcal{E}_0[x \mapsto l_1]$
 $\mathcal{E}_3 = \mathcal{E}_0[x \mapsto l_1, y \mapsto l_2]$

Question 17

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using mixed scope rules with *dynamic variable bindings* and *static procedure name bindings* (enlarge the size of the window of your browser for better visualisation of the derivation):

$$\begin{array}{c}
 \sigma_1 \circ \mathcal{E}_1 \vdash \underline{3} \rightarrow_E 3 \\
 \hline
 [\text{assBS}] \frac{}{\mathcal{E}_1, \Pi_3 \vdash_{l_1} \langle x := \underline{3}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \hline
 [\text{procBS}] \frac{}{\mathcal{E}_1, \Pi_2 \vdash_{l_1} \langle \text{proc } p \text{ is } x := \underline{2}; x := \underline{3}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \hline
 [\text{blockBS}] \frac{}{\mathcal{E}_1, \Pi_1 \vdash_{l_1} \langle \text{begin proc } p \text{ is } x := \underline{2}; x := \underline{3} \text{ end}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \hline
 [\text{procBS}] \frac{}{\mathcal{E}_1, \Pi_0 \vdash_{l_1} \langle \text{proc } p \text{ is } x := x + \underline{1}; \text{begin proc } p \text{ is } x := \underline{2}; x := \underline{3} \text{ end}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \hline
 [\text{decBS}] \frac{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{1} \rightarrow_E 1}{\mathcal{E}_0, \Pi_0 \vdash_{l_0} \langle \text{var lnt } x := \underline{1}; \text{proc } p \text{ is } x := x + \underline{1}; \text{begin proc } p \text{ is } x := \underline{2}; x := \underline{3} \text{ end}, \sigma_0 \rangle \rightarrow \sigma_2}
 \end{array}$$

where $l_{i+1} = \text{nxt}(l_i)$ and

$$\begin{array}{ll}
 \sigma_0 = \sigma & \mathcal{E}_0 = \emptyset \\
 \sigma_1 = \sigma[l_0 \mapsto 1] & \mathcal{E}_1 = \mathcal{E}[x \mapsto l_0] \\
 \sigma_2 = \sigma[l_0 \mapsto 3] &
 \end{array}$$

Provide the definitions of the procedure environments listed below by filling in the blanks so that the above is a well-formed derivation.

$$\Pi_0 = \emptyset \Pi$$

$$\Pi_1 = [p \mapsto x := x+1] \Pi \quad \checkmark$$

$$\Pi_2 = \emptyset [p \mapsto x := x+1] \Pi \quad \checkmark$$

$$\Pi_3 = [p \mapsto x := 2][p \mapsto x := x+1] \Pi \quad \checkmark$$

Question 18

[Flag question](#)

Correct

Mark 15.00 out of 15.00

Consider the typed Bip statement below:

```
var Int x := 3;  
var Int y := 2;  
proc p is (x := y - 1; y := x + 1);  
proc q is call p;  
begin  
  var Int x := 7;  
  proc p is x := x + 2;  
  call q;  
  y := x  
end
```

What is the final value assigned to the (global) variable y under the following scope rules?

- *Fully-static scope rule* (static variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rule* (dynamic variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rule* (static variable bindings and dynamic procedure name bindings): $y =$ ✓
- *Fully-dynamic scope rule* (dynamic variable bindings and dynamic procedure name bindings): $y =$ ✓

**Solution:**

- *Fully-static scope rule* (static variable bindings and static procedure name bindings): $y = 7$
- *Mixed scope rule* (dynamic variable bindings and static procedure name bindings): $y = 1$
- *Mixed scope rule* (static variable bindings and dynamic procedure name bindings): $y = 9$
- *Fully-dynamic scope rule* (dynamic variable bindings and dynamic procedure name bindings): $y = 9$

[Finish review](#)



Status

Finished

Started

Sunday, 15 June 2025, 2:42 PM

Completed

Sunday, 15 June 2025, 4:14 PM

Duration

1 hour 32 mins

Grade

105.00 out of 130.00 (80.77%)

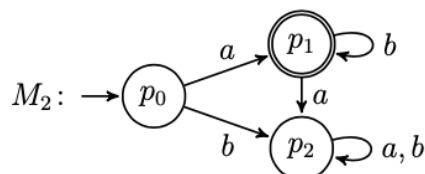
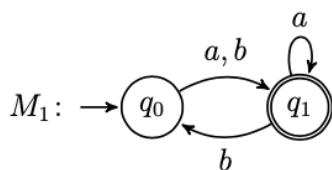
Question 1

Flag question

Correct

Mark 5.00 out of 5.00

Let M_1 and M_2 be the finite deterministic automata below, recognising the languages $L(M_1) = (a \cup b)(a \cup b(a \cup b))^*$ and $L(M_2) = ab^*$, respectively.

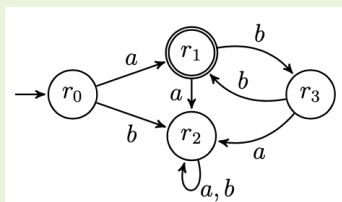


Which of the following automata recognises $L(M_1) \cap L(M_2)$ (the intersection of $L(M_1)$ and $L(M_2)$)?

Select one:

☒

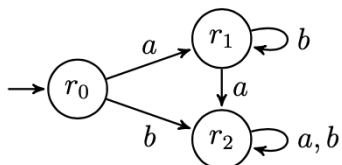
a.



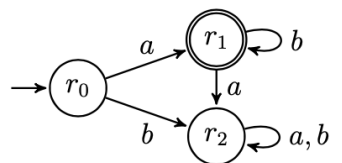
✓

☐

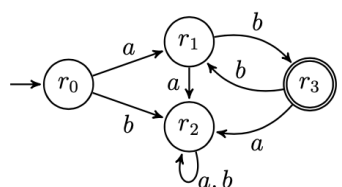
b.

☐

c.

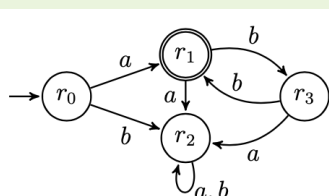
☐

d.



Your answer is correct.

The correct answer is:



Question 2

Flag question

Not answered

Marked out of 10.00

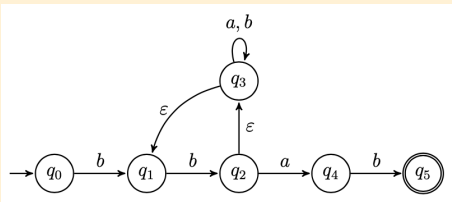
"Code-draw" the state diagram of a finite automaton recognising the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has prefix } bb \text{ and suffix } bab\}$$

The syntax of reference to code-draw the automaton is that of [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

☺ A possible solution to the exercise is:



#states

q0

q1

q2

q3

q4

q5

#initial

q0

#accepting

q5

#alphabet

a

b

#transitions

q0:b>q1

q1:b>q2

q2:a>q4

q2:\$>q3

q3:a>q3

q3:b>q3

q3:\$>q1

q4:b>q5

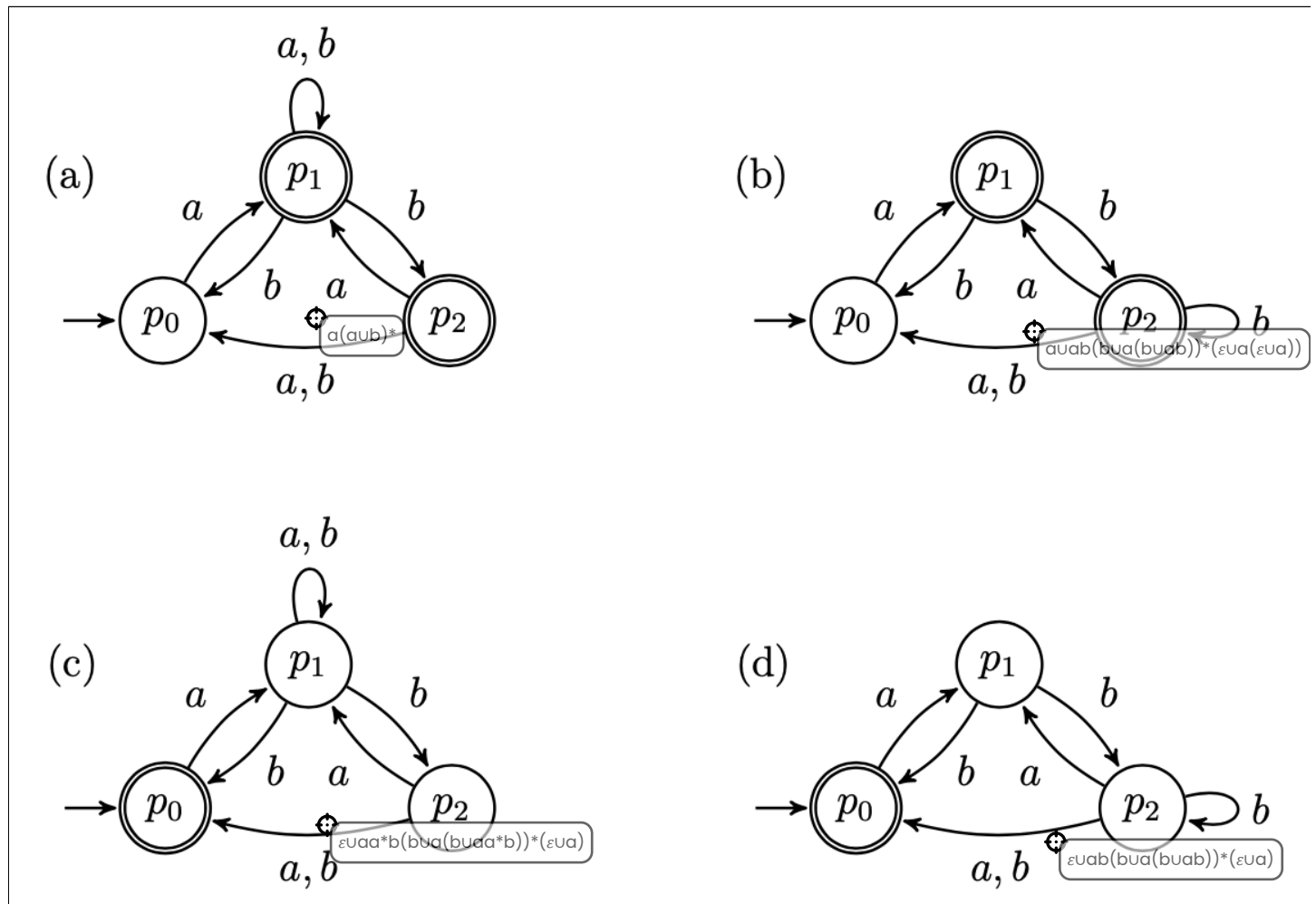
Question 3

Flag question

Correct

Mark 5.00 out of 5.00

Drag each regular expression onto the equivalent automaton.



😊 Your answer is correct.

The correct answer is:

(a) -- $a(a \cup b)^*$ (b) -- $a \cup ab(bua(buab))^*(\epsilon ua(\epsilon ua))$ (c) -- $\epsilon ua a^* b(bua(bua a^* b))^*(\epsilon ua)$ (d) -- $\epsilon ua b(bua(buab))^*(\epsilon ua)$

Question 4

Flag question

Not answered

Marked out of 15.00

By using the Pumping Lemma, prove that the language $L = \{(ab)^n a^m \mid n, m \geq 0 \text{ and } n > m\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking  to expand the editor console, and then  to select the equation editor).

 Solution:

To show that L is nonregular we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to choose $w = (ab)^{p+1}a^p$. (Note that $w \in L$ and $|w| = 2(p+1) + p \geq p$. So we are playing according to the rules of the game).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$, and $|xy| \leq p$
4. For any decomposition of w into xyz as above, we show that for $i = 0$, $xy^iz \notin L$.

Note that by the condition $|xy| \leq p$ we have that y occurs in the first half of $(ab)^{p+1}$ and z must contain at least an occurrence of b .

(CASE 1) y has an occurrence of ab . Then it means that in xy^0z , ab is consecutively repeated at most p -times. Thus $xy^iz \notin L$.

(CASE 2) y has no occurrences of ab . Then, as y is nonempty and occurs in $(ab)^{p+1}$, only three cases are possible: $y = ba$, $y = a$, or $y = b$. For each of these cases $xy^0z \notin (ab)^*a^*$, thus is not in the required format to belong in L .

Question 5

Flag question

Correct

Mark 5.00 out of 5.00

Select the correct option for each of the following languages:

- $\{a^n b^m \mid n, m \geq 0\}$: Regular  
- $\{a^n b^m \mid n, m \geq 0 \text{ and } n \geq m\}$: Nonregular  
- $\{a^n b^m a^n \mid n, m \geq 0\}$: Nonregular  
- $\{a^n b^m b^n \mid n, m \geq 0\}$: Nonregular  
- $\{b^n b^m \mid n, m \geq 0 \text{ and } n \neq m\}$: Regular  

 Solution:

- $\{a^n b^m \mid n, m \geq 0\}$: Regular
- $\{a^n b^m \mid n, m \geq 0 \text{ and } n \geq m\}$: Nonregular
- $\{a^n b^m a^n \mid n, m \geq 0\}$: Nonregular
- $\{a^n b^m b^n \mid n, m \geq 0\}$: Nonregular
- $\{b^n b^m \mid n, m \geq 0 \text{ and } n \neq m\}$: Regular

Question 6

Flag question

Correct

Mark 5.00 out of 5.00

Complete the context-free grammar below so that it generates the language $\{a^{2n}b^{2(n+1)} \mid n \geq 0\}$.

$S \rightarrow$ ☒ ☒ S ☒ ☒ $|$ ☒ ☒

😊 Your answer is correct.

Solution

$S \rightarrow aaSbb \mid bb$

Complete the context-free grammar below so that it generates the language $\{a^{2n}b^{2(n+1)} \mid n \geq 0\}$.
 The correct answer is: $S \rightarrow [a][a] S[b][b] \mid [b][b]$

Question 7

Flag question

Correct

Mark 5.00 out of 5.00

Let G be the context-free grammar below:

$S \rightarrow BAB$

$A \rightarrow abA \mid \varepsilon$

$B \rightarrow BbB \mid \varepsilon$

Are the following languages generated by G ?

- $\{w \mid w \in \{a, b\}^* \text{ where for every occurrence of } a \text{ there is an occurrence of } b\}$: ☒
- $\{w \mid w \in \{a, b\}^* \text{ where every occurrence of } a \text{ is followed by a } b\}$: ☒
- $b^*(ab)^*b^*$: ☒
- $b^* \cup (a \cup b)^* \cup b^*$: ☒
- $\{w \mid w \in \{a, b\}^* \text{ where there are more occurrences of } a\text{'s than } b\text{'s}\}$: ☒

Question 8

Flag question

Correct

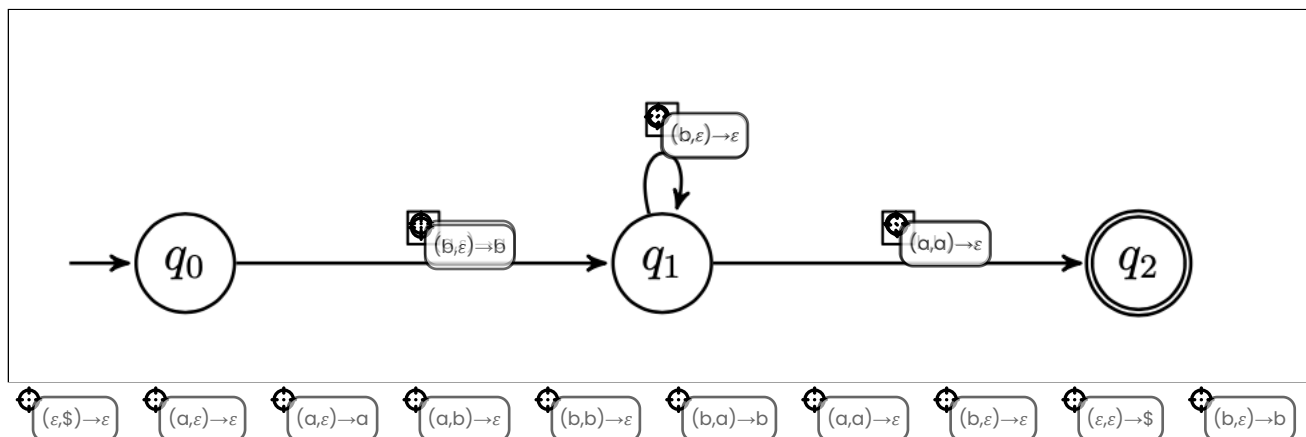
Mark 10.00 out of 10.00

Fill in the missing labels in the push-down automaton below so that it generates the language

$L = \{w \mid w \in \{a, b\}^* \text{ starts and ends with the same symbol}\}.$

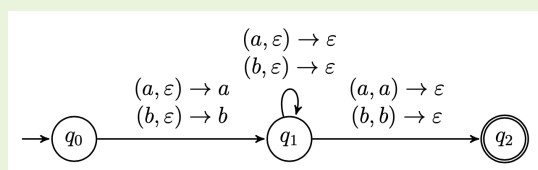
Instructions:

- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



😊 Your answer is correct.

Solution:



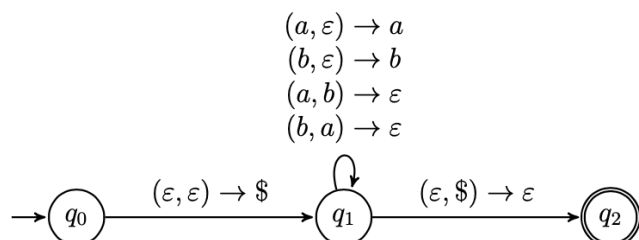
Question 9

Flag question

Correct

Mark 5.00 out of 5.00

Let M be the push-down automaton below:



Are the following languages recognised by M ?

- $\{w \mid w \in \{a, b\}^* \text{ has equal occurrences of } a\text{'s and } b\text{'s}\} :$ Yes ☒
- $\{a^n b^n \mid n \geq 0\} \cup \{b^n a^n \mid n \geq 0\} :$ No ☒
- $\{a^n b^n a^n b^n \mid n \geq 0\} :$ No ☒
- $\{(a \cup b)^n (a \cup b)^n \mid n \geq 0\} :$ No ☒
- $\{w \mid w \text{ has strictly more occurrences of } a\text{'s than } b\text{'s}\} :$ No ☒

Question 10

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derived Bims expressions:

true = $\underline{0} \leq \underline{0}$,

false = $\neg \text{true}$.

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree for the evaluation of **false** according to the big-step operational semantics for typed Bims expressions.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

[numBS]

$\underline{0} \rightarrow_{\text{Int}} \underline{0}$

$s \vdash \underline{0} \rightarrow_E \underline{0}$

[leqBS]

[numBS]

$\underline{0} \rightarrow_{\text{Int}} \underline{0}$

$s \vdash \underline{0} \rightarrow_E \underline{0}$

$s \vdash \text{true} \rightarrow_E tt$

[negBS]

$s \vdash \text{false} \rightarrow_E ff$

$\underline{0} \rightarrow_{\text{Int}} \underline{0}$

$s \vdash \underline{0} \rightarrow_E \underline{0}$

$s \vdash \text{true} \rightarrow_E tt$

$s \vdash \text{false} \rightarrow_E ff$

$s \vdash \underline{0} \rightarrow_E ff$

$s \vdash \underline{0} \rightarrow_E tt$

$s \vdash \text{true} \rightarrow_E ff$

$s \vdash \text{false} \rightarrow_E tt$

[numBS]

[leqBS]

[negBS]

[andBS]

😊 Your answer is correct.

Question 11

Flag question

Correct

Mark 10.00 out of 10.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the *state model* big-step operational semantics for typed Bims statements, where

 $IF = \text{if } x \leq 5 \text{ then } x := 3 \text{ else } x := 4,$
 $s_1 = s_0[x \mapsto 2],$
 $s_2 = s_0[x \mapsto 3],$
 $s_3 = s_0[x \mapsto 4].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$s_0 \vdash 2 \rightarrow_E 2$
 $\langle x := 2, s_0 \rangle \rightarrow s_1$

[ass_{BS}]

$s_1 \vdash 3 \rightarrow_E 3$
 $\langle x := 3, s_1 \rangle \rightarrow s_2$

[if-tt_{BS}]

$s_1 \vdash x \leq 5 \rightarrow_E tt$
 $\langle IF, s_1 \rangle \rightarrow s_2$

[comp_{BS}]

$\langle x := 2; IF, s_0 \rangle \rightarrow s_2$

$\langle IF, s_1 \rangle \rightarrow s_3$

$s_1 \vdash x \leq 5 \rightarrow_E ff$

$s_1 \vdash 4 \rightarrow_E 4$

$s_1 \vdash x \leq 5 \rightarrow_E tt$

$s_1 \vdash 3 \rightarrow_E 3$

$\langle x := 4, s_1 \rangle \rightarrow s_3$

$\langle IF, s_1 \rangle \rightarrow s_2$

$\langle x := 3, s_1 \rangle \rightarrow s_2$

[if-tt_{BS}]

[if-ff_{BS}]

[comp_{BS}]

[dec_{BS}]

[ass_{BS}]

Your answer is correct.

Question 12

Flag question

Correct

Mark 10.00 out of 10.00

Let $W = \text{while } x \leq 9 \text{ do } x := x + 3; y := x - 1$. Fill in the blanks in the derivation sequence below according to the rules of the state model small-step semantics for typed Bims statements. (Expand the window of your browsers for better visualisation).

$\langle W, [x \mapsto 0, y \mapsto 0] \rangle$
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; y := x - 1; W \text{ else skip}, [x \mapsto 0, y \mapsto 0] \rangle \quad (\text{while}_{SS})$
 $\Rightarrow \langle x := x + 3; y := x - 1; W, [x \mapsto 3, y \mapsto 0] \rangle \quad (\text{if-tt}_{SS})$
 $\Rightarrow \langle y := x - 1; W, [x \mapsto 3, y \mapsto 0] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle W, [x \mapsto 3, y \mapsto 2] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; y := x - 1; W \text{ else skip}, [x \mapsto 3, y \mapsto 2] \rangle \quad (\text{while}_{SS})$
 $\Rightarrow \langle x := x + 3; y := x - 1; W, [x \mapsto 6, y \mapsto 2] \rangle \quad (\text{if-tt}_{SS})$
 $\Rightarrow \langle y := x - 1; W, [x \mapsto 6, y \mapsto 2] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle W, [x \mapsto 6, y \mapsto 5] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; y := x - 1; W \text{ else skip}, [x \mapsto 6, y \mapsto 5] \rangle \quad (\text{while}_{SS})$
 $\Rightarrow \langle x := x + 3; y := x - 1; W, [x \mapsto 9, y \mapsto 5] \rangle \quad (\text{if-tt}_{SS})$
 $\Rightarrow \langle y := x - 1; W, [x \mapsto 9, y \mapsto 5] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle W, [x \mapsto 9, y \mapsto 8] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; y := x - 1; W \text{ else skip}, [x \mapsto 9, y \mapsto 8] \rangle \quad (\text{while}_{SS})$
 $\Rightarrow \langle x := x + 3; y := x - 1; W, [x \mapsto 12, y \mapsto 8] \rangle \quad (\text{if-tt}_{SS})$
 $\Rightarrow \langle y := x - 1; W, [x \mapsto 12, y \mapsto 8] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle W, [x \mapsto 12, y \mapsto 11] \rangle \quad (\text{ass}_{SS} + \text{comp-2}_{SS})$
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; y := x - 1; W \text{ else skip}, [x \mapsto 12, y \mapsto 11] \rangle \quad (\text{while}_{SS})$
 $\Rightarrow \langle \text{skip}, [x \mapsto 12, y \mapsto 11] \rangle \quad (\text{if-ff}_{SS})$
 $\Rightarrow [x \mapsto 12, y \mapsto 11] \quad (\text{skip}_{SS})$

Question 13

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the type system for typed Bims statements, where

 $DEC = \text{var Int } y := x + \underline{1}; \text{skip},$
 $E_1 = [x \mapsto \text{Int}],$
 $E_2 = [x \mapsto \text{Int}, y \mapsto \text{Int}].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$y \notin \text{dom}(E_1)$
 $[\text{dec}_{\text{TS}}] \frac{}{E_1 \vdash DEC: \text{ok}}$

$E_1 \vdash x + \underline{1}: \text{Int}$

$E_2 \vdash \text{skip}: \text{ok}$

$E_1(x) = \text{Int}$
 $[\text{var}_{\text{TS}}] \frac{}{E_1 \vdash x: \text{Int}}$

$E_1 \vdash 1: \text{Int}$
 $[\text{num}_{\text{TS}}] \frac{}{E_1 \vdash 1: \text{Int}}$

$E_2 \vdash \text{skip}: \text{ok}$
 $[\text{skip}_{\text{TS}}] \frac{}{E_2 \vdash \text{skip}: \text{ok}}$

$E_1 \vdash x + \underline{1}: \text{Int}$

$E_1 \vdash x: \text{Int}$

$E_1 \vdash 1: \text{Int}$

$E_1(x) = \text{Int}$

$E_2 \vdash \text{skip}: \text{ok}$

$E_2 \vdash x + \underline{1}: \text{Int}$

$E_2 \vdash x: \text{Int}$

$E_2 \vdash 1: \text{Int}$

$E_1 \vdash \text{skip}: \text{ok}$

$E_2(x) = \text{Int}$

$E_2(y) = \text{Int}$

$[\text{sum}_{\text{TS}}]$

$[\text{var}_{\text{TS}}]$

$[\text{num}_{\text{TS}}]$

$[\text{skip}_{\text{TS}}]$

$[\text{dec}_{\text{TS}}]$

Your answer is correct.

Question 14

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation-tree constructed according to the rules of the type system for Bims statements with typed variable declarations.

$$\begin{array}{c}
 \begin{array}{c}
 x \notin \text{dom}(\emptyset) \\
 [\text{dec}_{\text{TS}}] \frac{}{}
 \end{array}
 \quad
 \begin{array}{c}
 \begin{array}{c}
 \text{num}_{\text{TS}} \\
 \frac{}{}
 \end{array}
 \quad
 \begin{array}{c}
 \text{dec}_{\text{TS}} \\
 \frac{}{}
 \end{array}
 \quad
 \begin{array}{c}
 \text{num}_{\text{TS}} \\
 \frac{}{}
 \end{array}
 \quad
 \begin{array}{c}
 \text{skip}_{\text{TS}} \\
 \frac{}{}
 \end{array}
 \end{array}
 \end{array}
 \frac{}{}
 \begin{array}{c}
 \emptyset \vdash 1: \text{Int} \quad E_1 \vdash \text{var Int } y := \underline{2}; \text{skip}: \text{ok} \\
 E_1 \vdash \text{var Int } x := \underline{1}; \text{var Int } y := \underline{2}; \text{skip}: \text{ok}
 \end{array}$$

Provide the definitions of the typing environments E_1 and E_2 so that the above is a well-formed derivation proving that the statement $\text{var Int } x := \underline{1}; \text{var Int } y := \underline{2}; \text{skip}$ is well-typed.

Select the appropriate values among the choices provided below:

 $E_1 = [x \mapsto \text{Int}, y \mapsto \text{undefined}]$
 $E_2 = [x \mapsto \text{Int}, y \mapsto \text{Int}]$


Question 15

Flag question

Correct

Mark 5.00 out of 5.00

Choose the correct assertion about the following Bims statements and expressions:

- `var Bool b; if b then b :=  $\neg b$  else b := true` is

☐

ill-typed, because the variable b is declared without an initial value

☐

well-typed

☒

ill-formed



Mark 1.00 out of 1.00

The correct answer is: ill-formed

- `var Int x := x + 1; var Int y := x; skip` is

☒

ill-typed because an undeclared variable is used in an assignment


☐

ill-typed because x is undeclared

☐

well-typed

Mark 1.00 out of 1.00

The correct answer is: ill-typed because an undeclared variable is used in an assignment

- `var Int x := 0; x := x` is

☐

ill-typed because the self-recursive assignment does not terminate

☒

well-typed


☐

ill-formed because the assignment $x:=x$ is self-recursive

Mark 1.00 out of 1.00

The correct answer is: well-typed

- `var Int x := 0; var Int x := 0; skip` is

☒

ill-typed because the variable x is declared twice


☐

ill-formed

☐

well-typed

Mark 1.00 out of 1.00

The correct answer is: ill-typed because the variable x is declared twice

- `var Int x := 0; var Bool z := 0 ≤ x; skip` is

☐

ill-typed because the variable z is assigned an integer value although it is declared as Boolean

☐

ill-formed



well-typed



Mark 1.00 out of 1.00

The correct answer is: well-typed

Question 16

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using fully-dynamic scope rules

$$\begin{array}{c}
 \frac{\sigma_1 \circ \mathcal{E}_1 \vdash x + \underline{1} \rightarrow_E 2}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle x := x + \underline{1}, \sigma_1 \rangle \rightarrow \sigma_2} [\text{assBS}] \quad \frac{\sigma_2 \circ \mathcal{E}_1 \vdash \underline{2} \rightarrow_E 2 \quad \mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{skip}, \sigma_3 \rangle \rightarrow \sigma_3}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var lnt } x := \underline{2}; \text{skip}, \sigma_2 \rangle \rightarrow \sigma_3} [\text{decBS}] \\
 \frac{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle x := x + \underline{1}, \sigma_1 \rangle \rightarrow \sigma_2 \quad \mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var lnt } x := \underline{2}; \text{skip}, \sigma_2 \rangle \rightarrow \sigma_3}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle x := x + \underline{1}; \text{var lnt } x := \underline{2}; \text{skip}, \sigma_1 \rangle \rightarrow \sigma_3} [\text{compBS}] \\
 \frac{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{1} \rightarrow_E 1 \quad \mathcal{E}_1, \pi_0 \vdash_{l_1} \langle x := x + \underline{1}; \text{var lnt } x := \underline{2}; \text{skip}, \sigma_1 \rangle \rightarrow \sigma_3}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{begin } x := x + \underline{1}; \text{var lnt } x := \underline{2}; \text{skip end}, \sigma_1 \rangle \rightarrow \sigma_3} [\text{blockBS}] \\
 \frac{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{1} \rightarrow_E 1 \quad \mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{begin } x := x + \underline{1}; \text{var lnt } x := \underline{2}; \text{skip end}, \sigma_1 \rangle \rightarrow \sigma_3}{\mathcal{E}_0, \pi_0 \vdash_{l_0} \langle \text{var lnt } x := \underline{1}; \text{begin } x := x + \underline{1}; \text{var lnt } x := \underline{2}; \text{skip end}, \sigma_0 \rangle \rightarrow \sigma_3} [\text{decBS}]
 \end{array}$$

where $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the size of the window of your browser for better visualisation of the derivation).

Provide the definitions of the stores and variable environments listed below by filling in the blanks so that the above is a well-formed derivation.

 $\sigma_0 = \emptyset$
 $\sigma_1 = [l_0 \mapsto \text{1}, l_1 \mapsto \text{undefined}, l_2 \mapsto \text{undefined}]$
 $\sigma_2 = [l_0 \mapsto \text{2}, l_1 \mapsto \text{undefined}, l_2 \mapsto \text{undefined}]$
 $\sigma_3 = [l_0 \mapsto \text{2}, l_1 \mapsto \text{2}, l_2 \mapsto \text{undefined}]$
 $\mathcal{E}_1 = \mathcal{E}_0[x \mapsto \text{L0}]$
 $\mathcal{E}_2 = \mathcal{E}_0[x \mapsto \text{L1}]$

Question 17

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using mixed scope rules with *dynamic variable bindings* and *static procedure name bindings* (enlarge the size of the window of your browser for better visualisation of the derivation):

$$\begin{array}{c}
 \text{[skipBS]} \frac{}{\mathcal{E}_2, \Pi_2 \vdash_{l_2} \langle \text{skip}, \sigma_2 \rangle \rightarrow \sigma_2} \\
 \text{[procBS]} \frac{}{\mathcal{E}_2, \Pi_1 \vdash_{l_2} \langle \text{proc } p \text{ is } x := \underline{2}; \text{skip}, \sigma_2 \rangle \rightarrow \sigma_2} \\
 \text{[decBS]} \frac{\sigma_1 \circ \mathcal{E}_1 \vdash \underline{0} \rightarrow_E 0}{\mathcal{E}_1, \Pi_1 \vdash_{l_1} \langle \text{var lnt } x := \underline{0}; \text{proc } p \text{ is } x := \underline{2}; \text{skip}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \text{[blockBS]} \frac{}{\mathcal{E}_1, \Pi_0 \vdash_{l_1} \langle \text{begin var lnt } x := \underline{0}; \text{proc } p \text{ is } x := \underline{2}; \text{skip end}, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \text{[decBS]} \frac{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{1} \rightarrow_E 1}{\mathcal{E}_0, \Pi_0 \vdash_{l_0} \langle \text{var lnt } x := \underline{1}; \text{begin var lnt } x := \underline{0}; \text{proc } p \text{ is } x := \underline{2}; \text{skip end}, \sigma_0 \rangle \rightarrow \sigma_2}
 \end{array}$$

where $l_{i+1} = \text{nxt}(l_i)$ and

$$\begin{array}{ll}
 \sigma_0 = \sigma & \mathcal{E}_0 = \emptyset \\
 \sigma_1 = \sigma[l_0 \mapsto 1] & \mathcal{E}_1 = \mathcal{E}[x \mapsto l_0] \\
 \sigma_2 = \sigma[l_0 \mapsto 1, l_1 \mapsto 0] & \mathcal{E}_2 = \mathcal{E}[x \mapsto l_1]
 \end{array}$$

Provide the definitions of the procedure environments listed below by filling in the blanks so that the above is a well-formed derivation.

$\Pi_0 = \emptyset \Pi$

$\Pi_1 =$ ✓

$\Pi_2 =$ ✓

Question 18

Flag question

Correct

Mark 15.00 out of 15.00

Consider the typed Bip statement below:

```

var lnt x := 1;
var lnt y := 2;
proc p is (x := x + 2; y := x + 1);
proc q is call p;
begin
  var lnt x := 3;
  proc p is x := x - 1;
  call q;
  y := x
end

```

What is the final value assigned to the (global) variable y under the following scope rules?

- Fully-static scope rule (static variable bindings and static procedure name bindings): $y =$ ✓
- Mixed scope rule (dynamic variable bindings and static procedure name bindings): $y =$ ✓
- Mixed scope rule (static variable bindings and dynamic procedure name bindings): $y =$ ✓
- Fully-dynamic scope rule (dynamic variable bindings and dynamic procedure name bindings): $y =$ ✓

Finish review



Status
Finished

Started
Monday, 16 June 2025, 12:58 PM

Completed
Monday, 16 June 2025, 1:09 PM

Duration
10 mins 51 secs

Grade
92.50 out of 130.00 (71.15%)

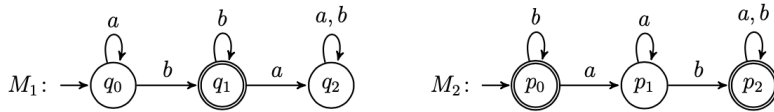
Question 1

Flag question

Correct

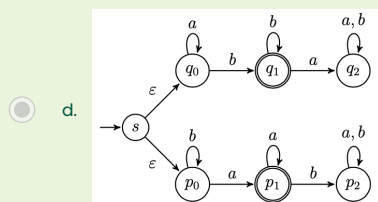
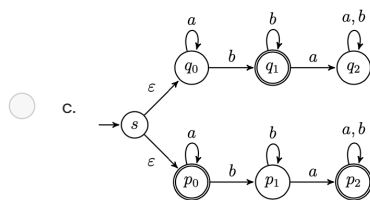
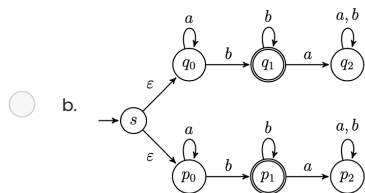
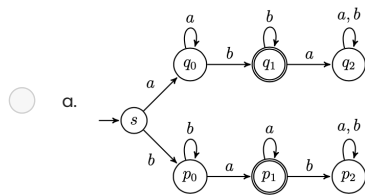
Mark 5.00 out of 5.00

Let M_1 and M_2 be the finite automata below:



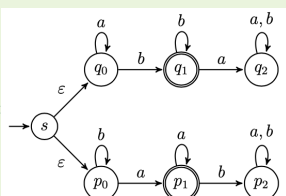
Which of the following automata recognise the language $L(M_1) \cup \overline{L(M_2)}$ (the union of the language recognised by M_1 and the complement of the language recognised by M_2)?

Select one:



😊 Your answer is correct.

The correct answer is:



Question 2

Flag question

Not answered

Marked out of 10.00

"Code-draw" the state diagram of a **nondeterministic finite automaton with at most 10 states** that recognises the language

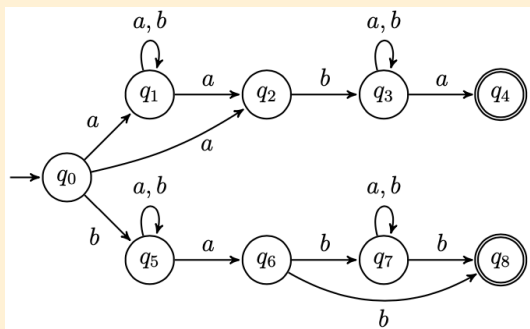
$$L = \{w \in \{a, b\}^* \mid w \text{ starts and end with the same symbol and contains } ab\}.$$

Observe that any solution with less than 10 states is also accepted.

The syntax of reference to code-draw the automaton is that of the [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look at how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

Solution:



#states

q0

q1

q2

q3

q4

q5

q6

q7

q8

#initial

q0

#accepting

q4

q8

#alphabet

a

b

#transitions

q0:a>q1

q0:b>q5

q0:a>q2

q1:a>q2

q1:a>q1

q1:b>q1

q2:b>q3
 q3:a>q4
 q3:a>q3
 q3:b>q3
 q5:a>q6
 q5:a>q5
 q5:b>q5
 q6:b>q7
 q6:b>q8
 q7:b>q8
 q7:a>q7
 q7:b>q7

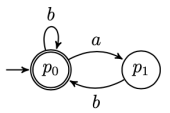
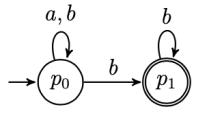
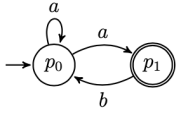
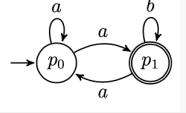
Question 3

Flag question

Not answered

Marked out of 5.00

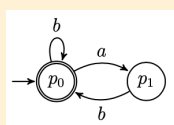
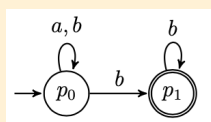
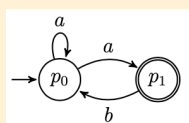
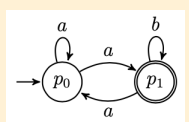
Drag the regular expressions next to the corresponding equivalent automaton.

	Drag answer here
	Drag answer here
	Drag answer here
	Drag answer here

 $(a^* \cup b^*)bb^*$ $(ab)^*a \cup b^*$ $(ab \cup b)^*$ $(a^*ab)^*a^*a$ $(a \cup ab^*a)^*ab^*$ $(a \cup b)^*bb^*$ $(ab)^* \cup b^*$

☹ Your answer is incorrect.

The correct answer is:

 $(ab \cup b)^*$  $(a \cup b)^*bb^*$  $(a^*ab)^*a^*a$  $(a \cup ab^*a)^*ab^*$

Question 4

Flag question

Not answered

Marked out of 15.00

By using the Pumping Lemma, prove that the language $L = \{(aba)^n b^n \mid n \geq 0\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking the symbol $\equiv$ to expand the editor console, and then $\times$ to select the equation editor).

Solution:

To show that L is nonregular we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to choose $w = (aba)^p b^p$. (Note that $w \in L$ and $|w| = 4p \geq p$. So we are playing according to the rules of the game).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$, and $|xy| \leq p$
4. For any decomposition of w into xyz as above, we show that for $i = 0$, $xy^i z \notin L$.
Observe that since $y \neq \varepsilon$, demon must have chosen $p > 0$. Moreover, by the condition $|xy| \leq p$ we also know that xy must occur within $(aba)^p$ and z must have b^p as suffix and contain at least an a .
(CASE 1) y has an occurrence of b . Then, it means that in xy^0 the symbol b occurs at most $(p-1)$ -times. Thus, $xy^0 z \notin L$, because the number of occurrences of b 's in the first part of the string do not match that appearing in the suffix b^p .
(CASE 2) y has no occurrences of b . Then, $y = a$ or $y = aa$. In this case $xy^0 z \notin L$ because it is not in the form $(aba)^* b^*$.

Question 5

Flag question

Correct

Mark 5.00 out of 5.00

Select the *most precise* option for each of the following languages:

- $\{w \mid w \in \{a, b\}^* \text{ starts and ends with a different symbol}\}$: Regular ☒
- $\{w^3 \mid w \in \{a, b\}^*\}$: Not context-free ☒
- $\{wb^n \mid w \in \{a, b\}^* \text{ and } |w| = n\}$: Context-free ☒
- $\{a^n ba^{2m} \mid n, m \geq 0\}$: Regular ☒
- $\{(ab)^n a^n \mid n \geq 0\}$: Context-free ☒
- $\{(ab)^n a^n b^n \mid n \geq 0\}$: Not context-free ☒

Solution:

- $\{w \mid w \in \{a, b\}^* \text{ starts and ends with a different symbol}\}$: Regular (It is equivalent to the regular expression $a(a \cup b)^* b \cup b(a \cup b)^* a$).
- $\{w^3 \mid w \in \{a, b\}^*\}$: Not context-free (It can be proven using the Pumping lemma for context-free languages. The proof is similar to Example 2.38 (pg. 129) in Michael Sipser's book *Introduction to the Theory of Computation*)
- $\{wb^n \mid w \in \{a, b\}^* \text{ and } |w| = n\}$: Context-free (It is generated by the grammar $S \rightarrow \varepsilon \mid aSb \mid bSb$)
- $\{a^n ba^{2m} \mid n, m \geq 0\}$: Regular (It is equivalent to the regular expression $a^* b(aa)^*$)
- $\{(ab)^n a^n \mid n \geq 0\}$: Context-free (It is generated by the grammar $S \rightarrow \varepsilon \mid abSa$)
- $\{(ab)^n a^n b^n \mid n \geq 0\}$: Not context-free (It can be proven using the Pumping lemma for context-free languages. The proof is similar to Example 2.36 (pg. 128) in Michael Sipser's book *Introduction to the Theory of Computation*)

Question 6

Flag question

Not answered

Marked out of 5.00

Drag the languages next to the context-free grammar that generates them.

$$S \rightarrow \varepsilon \mid F$$

$$F \rightarrow aFb$$

Drag answer here

$$S \rightarrow FF$$

$$F \rightarrow \varepsilon \mid aFb$$

Drag answer here

$$S \rightarrow b \mid aSb \mid bSb$$

Drag answer here

$$S \rightarrow aSb \mid bSa$$

Drag answer here

 $\{w \mid w \in \{a, b\}^* \text{ has more } b\text{'s than } a\text{'s}\}$
 $\{w \mid w \in \{a, b\}^* \text{ has equally many } a\text{'s and } b\text{'s}\}$
 $\{(a^n b^n)^2 \mid n \geq 0\}$
 $\{wb^{n+1} \mid w \in \{a, b\}^* \text{ and } |w| = n\}$
 $\{a^n b^n a^m b^m \mid n, m > 0\}$
 $\emptyset$
 $\{a^n b^n a^m b^m \mid n, m \geq 0\}$
 $\{(ab)^n b^{n+1} \mid n \geq 0\}$
 $\{\varepsilon\}$
 $\{a^n b^n \mid n \geq 0\}$

☹ Your answer is incorrect.

The correct answer is:

$$S \rightarrow \varepsilon \mid F$$

$$F \rightarrow aFb$$
 $\{\varepsilon\}$

$$S \rightarrow FF$$

$$F \rightarrow \varepsilon \mid aFb$$
 $\{a^n b^n a^m b^m \mid n, m \geq 0\}$

$$S \rightarrow b \mid aSb \mid bSb$$
 $\{wb^{n+1} \mid w \in \{a, b\}^* \text{ and } |w| = n\}$

$$S \rightarrow aSb \mid bSa$$
 $\emptyset$

Question 7

Flag question

Partially correct

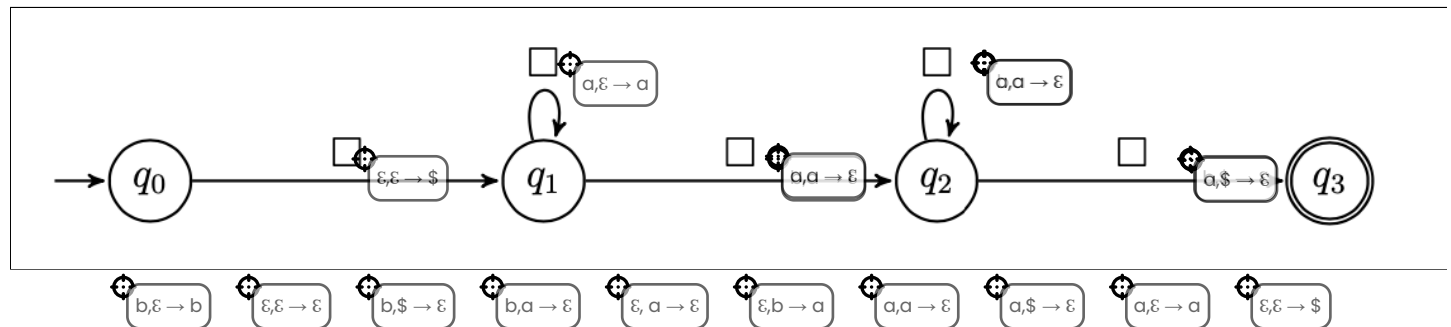
Mark 7.50 out of 10.00

Fill in the missing labels in the push-down automaton below so that it generates the language

$$L = \{a^n w \mid w \in \{a, b\}^* \text{ and } |w| = n + 1\}.$$

Instructions:

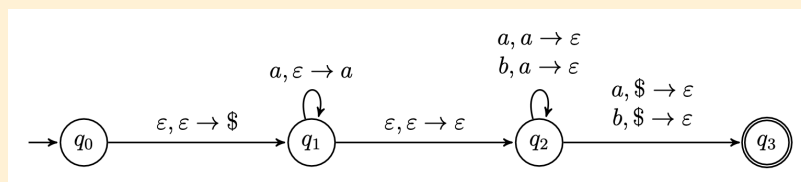
- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



☹️ Your answer is partially correct.

You have correctly selected 6.

Solution:



Question 8

Flag question

Correct

Mark 5.00 out of 5.00

Complete the context-free grammar below so that it generates the language

$$L = \{waw^R \mid w \in \{a, b\}^*\}$$

where w^R denotes the reverse of w , that is, the string obtained by writing w in the opposite order (for example, if $w = abaa$, then $w^R = aaba$).

$S \rightarrow$ ☒ | ☒ ☒ | ☒ | ☒ ☒

😊 **Solution:**

$$S \rightarrow a \mid aSa \mid bSb$$

Question 9

Flag question

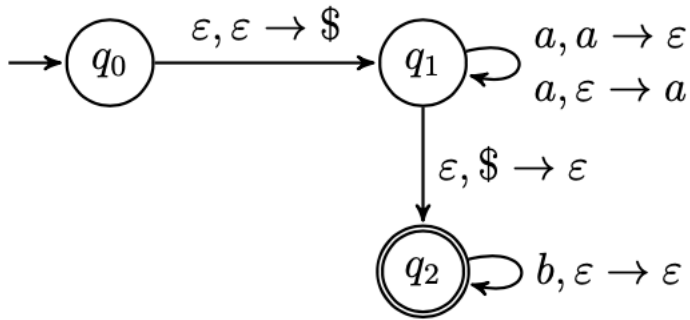
Correct

Mark 10.00 out of 10.00

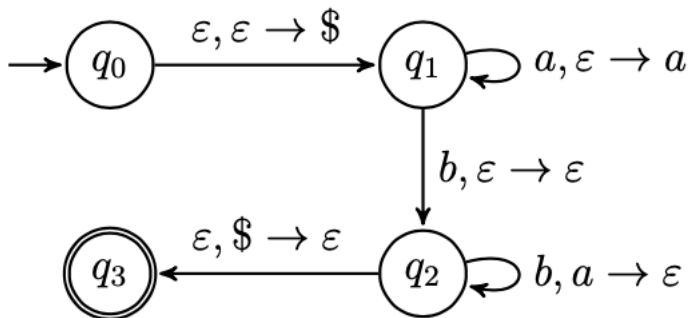
Match each pushdown automaton to the language it recognises.

Instructions:

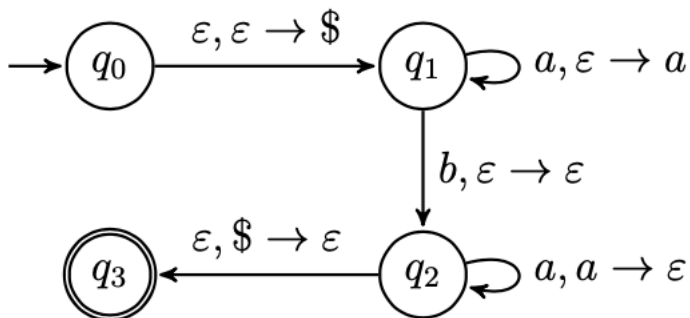
- Scroll down to see all the available options.
- Drag and drop the items into the boxes.
- The items can be used zero or more times.



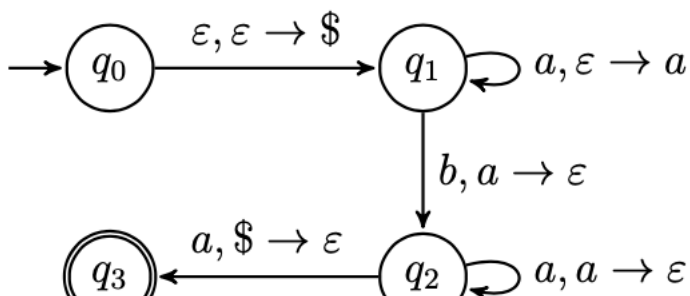
$$\{a^{2n}b^m \mid n, m \geq 0\}$$



$$\{a^n b^{n+1} \mid n \geq 0\}$$



$$\{a^n b a^n \mid n \geq 0\}$$



$$\{a^n b a^n \mid n > 0\}$$

$$\{a^n b^{n+1} \mid n > 0\}$$

$$\{a^{2n} b^m \mid n, m \geq 0\}$$

$$\{a^n b a^n \mid n \geq 0\}$$

$$\{a^n b^{n+1} \mid n \geq 0\}$$

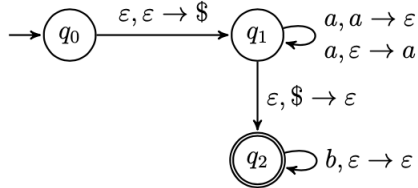
$$\{a^n b a^{n+1} \mid n \geq 0\}$$

$$\{a^n b^m \mid n, m \geq 0\}$$

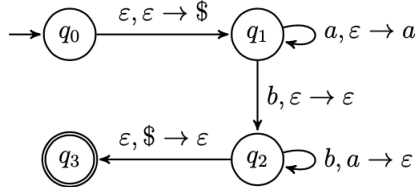
$$\{a^n b a^n \mid n > 0\}$$

😊 Your answer is correct.

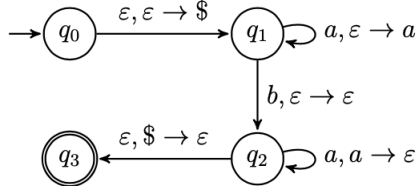
Solution:



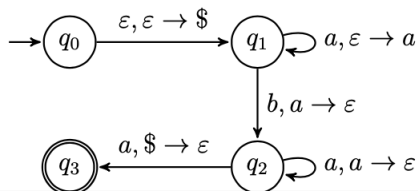
$$\{a^{2n}b^m \mid n, m \geq 0\}$$



$$\{a^n b^{n+1} \mid n \geq 0\}$$



$$\{a^n b a^n \mid n \geq 0\}$$



$$\{a^n b a^n \mid n > 0\}$$

Question 10

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree for the evaluation of $(\underline{2}x + \underline{3})y$ according to the big-step operational semantics for typed Bims expressions.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

Proof tree diagram showing the evaluation of $(\underline{2}x + \underline{3})y$ according to the big-step operational semantics for typed Bims expressions. The diagram consists of several boxes containing expressions and judgments, connected by lines representing the derivation steps. The boxes are arranged in a hierarchical structure, with the root expression $(\underline{2}x + \underline{3})y$ at the top. The boxes are labeled with the following expressions and judgments:

- $\underline{2} \rightarrow_{\text{Int}} 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s(x) = 2$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash \underline{2} \rightarrow_E 2$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash x \rightarrow_E 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s \vdash \underline{2}x \rightarrow_E 4$ (labeled $[\text{sum}_{\text{BS}}]$)
- $s \vdash \underline{2}x + \underline{3} \rightarrow_E 7$ (labeled $[\text{par}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3}) \rightarrow_E 7$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash y \rightarrow_E 3$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3})y \rightarrow_E 21$ (labeled $[\text{mult}_{\text{BS}}]$)
- $\underline{2} \rightarrow_{\text{Int}} 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s(x) = 2$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash \underline{2} \rightarrow_E 2$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash x \rightarrow_E 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s \vdash \underline{2}x \rightarrow_E 4$ (labeled $[\text{sum}_{\text{BS}}]$)
- $s \vdash \underline{2}x + \underline{3} \rightarrow_E 7$ (labeled $[\text{par}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3}) \rightarrow_E 7$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash y \rightarrow_E 3$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3})y \rightarrow_E 21$ (labeled $[\text{mult}_{\text{BS}}]$)

😊 Your answer is correct.

Solution:

Solution diagram showing the evaluation of $(\underline{2}x + \underline{3})y$ according to the big-step operational semantics for typed Bims expressions. The diagram consists of several boxes containing expressions and judgments, connected by lines representing the derivation steps. The boxes are arranged in a hierarchical structure, with the root expression $(\underline{2}x + \underline{3})y$ at the top. The boxes are labeled with the following expressions and judgments:

- $\underline{2} \rightarrow_{\text{Int}} 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s(x) = 2$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash \underline{2} \rightarrow_E 2$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash x \rightarrow_E 2$ (labeled $[\text{num}_{\text{BS}}]$)
- $s \vdash \underline{2}x \rightarrow_E 4$ (labeled $[\text{sum}_{\text{BS}}]$)
- $s \vdash \underline{2}x + \underline{3} \rightarrow_E 7$ (labeled $[\text{par}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3}) \rightarrow_E 7$ (labeled $[\text{mult}_{\text{BS}}]$)
- $s \vdash y \rightarrow_E 3$ (labeled $[\text{var}_{\text{BS}}]$)
- $s \vdash (\underline{2}x + \underline{3})y \rightarrow_E 21$ (labeled $[\text{mult}_{\text{BS}}]$)

Question 11

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the *state model* big-step operational semantics for typed Bims statements, where

 $W = \text{while } x \leq \underline{2} \text{ do } x := x + \underline{1},$
 $s_1 = s_0[x \mapsto 2],$
 $s_2 = s_0[x \mapsto 3],$
 $s_3 = s_0[x \mapsto 4].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

The diagram shows a proof tree structure with the following components:

- Top level:**
 - Left: $s_0 \vdash \underline{2} \rightarrow_E 2$ (with $[assBS]$ rule box)
 - Right: $s_1 \vdash x + \underline{1} \rightarrow_E 3$ (with $[assBS]$ rule box)
 - Far right: $s_2 \vdash x \leq \underline{2}$
- Second level:**
 - Left: $\langle x := \underline{2}, s_0 \rangle \rightarrow s_1$ (with $[compBS]$ rule box)
 - Middle: $\langle x := x + \underline{1}, s_1 \rangle \rightarrow s_2$ (with $[compBS]$ rule box)
 - Right: $\langle W, s_2 \rangle \rightarrow s_2$
- Third level:**
 - Left: $\langle x := \underline{2}, s_1 \rangle \rightarrow s_1$ (with $[compBS]$ rule box)
 - Middle: $s_2 \vdash x \leq \underline{2} \rightarrow_E ff$
 - Right: $\langle x := \underline{2}, s_0 \rangle \rightarrow s_1$
- Fourth level:**
 - Left: $\langle W, s_2 \rangle \rightarrow s_3$
 - Middle: $s_1 \vdash x \leq \underline{2} \rightarrow_E ff$
 - Right: $s_2 \vdash x \leq \underline{2} \rightarrow_E tt$
- Fifth level:**
 - Left: $\langle W, s_1 \rangle \rightarrow s_2$
 - Middle: $\langle W, s_1 \rangle \rightarrow s_1$
 - Right: $s_1 \vdash x + \underline{1} \rightarrow_E 3$
- Bottom level:**
 - Left: $\langle x := x + \underline{1}, s_1 \rangle \rightarrow s_2$
 - Middle: $[while-ffBS]$ rule box
 - Right: $[while-ttBS]$ rule box

😊 Your answer is correct.

Solution:

$$\begin{array}{c}
 \begin{array}{c}
 [assBS] \frac{s_0 \vdash \underline{2} \rightarrow_E 2}{\langle x := \underline{2}, s_0 \rangle \rightarrow s_1} \\
 [compBS] \frac{\langle x := \underline{2}, s_0 \rangle \rightarrow s_1}{\langle x := \underline{2}; W, s_0 \rangle \rightarrow s_2}
 \end{array}
 \quad
 \begin{array}{c}
 [assBS] \frac{s_1 \vdash x + \underline{1} \rightarrow_E 3}{\langle x := x + \underline{1}, s_1 \rangle \rightarrow s_2} \\
 [compBS] \frac{\langle x := x + \underline{1}, s_1 \rangle \rightarrow s_2}{\langle x := x + \underline{1}; W, s_1 \rangle \rightarrow s_2} \\
 [while-ttBS] \frac{\langle x := x + \underline{1}; W, s_1 \rangle \rightarrow s_2}{\langle W, s_1 \rangle \rightarrow s_2}
 \end{array}
 \quad
 \begin{array}{c}
 [while-ffBS] \frac{s_2 \vdash x \leq \underline{2} \rightarrow_E ff}{\langle W, s_2 \rangle \rightarrow s_2} \\
 [while-ffBS] \frac{\langle W, s_2 \rangle \rightarrow s_2}{\langle W, s_1 \rangle \rightarrow s_2}
 \end{array}
 \quad
 s_1 \vdash x \leq \underline{2} \rightarrow_E tt
 \end{array}$$

Question 12

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the state model big-step operational semantics for typed Bims statements. (Enlarge the window for a better view of the derivation).

$$\begin{array}{c}
 \text{[dec}_{\text{BS}}] \frac{s_0 \vdash \underline{2} \rightarrow_E 2}{\langle \text{var Int } x := \underline{2}; y := \underline{2}x; y := y + \underline{1}; x := x - y, s_0 \rangle \rightarrow s_4} \\
 \text{[comp}_{\text{BS}}] \frac{\text{[ass}_{\text{BS}}] \frac{s_1 \vdash \underline{2}x \rightarrow_E 4}{\langle y := \underline{2}x, s_1 \rangle \rightarrow s_2} \quad \text{[comp}_{\text{BS}}] \frac{\text{[ass}_{\text{BS}}] \frac{s_2 \vdash y + \underline{1} \rightarrow_E 5}{\langle y := y + \underline{1}, s_2 \rangle \rightarrow s_3} \quad \text{[ass}_{\text{BS}}] \frac{s_3 \vdash x - y \rightarrow_E -3}{\langle x := x - y, s_3 \rangle \rightarrow s_4}}{\langle y := y + \underline{1}; x := x - y, s_2 \rangle \rightarrow s_4}}
 \end{array}$$

Fill in the blanks in the definitions below so that the above is a well-formed derivation; use "-" if the value is undefined.

$$s_0 = \emptyset$$

$$s_1 = [x \mapsto 2, y \mapsto -]$$

$$s_2 = [x \mapsto 2, y \mapsto 4]$$

$$s_3 = [x \mapsto 2, y \mapsto 5]$$

$$s_4 = [x \mapsto -3, y \mapsto 5]$$

Question 13

Flag question

Correct

Mark 5.00 out of 5.00

Let

 $IF = \text{if } x \leq y \text{ then } x := x + 2 \text{ else } y := y - 2,$
 $W = \text{while } (x \leq 2 \vee 2 \leq y) \text{ do } IF.$

Fill in the blanks in the derivation sequence below according to the rules of the state model small-step semantics for typed Bims statements. (Expand the window of your browsers for better visualisation).

$\langle W, [x \mapsto 0, y \mapsto 2] \rangle$
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 0, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 0, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle x := x + 2; W, [x \mapsto 0, y \mapsto 2] \rangle$ (if-tt_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 2, y \mapsto 2] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 2, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 2, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle x := x + 2; W, [x \mapsto 2, y \mapsto 2] \rangle$ (if-tt_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 2] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 4, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 2; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-ff_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 0] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 4, y \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 4, y \mapsto 0] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 4, y \mapsto 0]$ (skip_{SS})

Solution:

$\langle W, [x \mapsto 0, y \mapsto 2] \rangle$
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 0, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 0, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle x := x + 2; W, [x \mapsto 0, y \mapsto 2] \rangle$ (if-tt_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 2, y \mapsto 2] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 2, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 2, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle x := x + 2; W, [x \mapsto 2, y \mapsto 2] \rangle$ (if-tt_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 2] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 4, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle IF; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := y - 2; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-ff_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 0] \rangle$ (ass_{SS}+comp-2_{SS})
 $\Rightarrow \langle \text{if } (x \leq 2 \vee 2 \leq y) \text{ then } IF; W \text{ else skip}, [x \mapsto 4, y \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 4, y \mapsto 0] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 4, y \mapsto 0]$ (skip_{SS})

Question 14

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the type system for typed Bims statements, where $E = [b \mapsto \text{Bool}]$.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$E \vdash \text{var Int } z := 0; b := z \leq 3; \text{ok}$

$E(z) = \text{Int}$	$E[z \mapsto \text{Int}](z) = \text{Int}$	$E \vdash 0 : \text{Int}$	$E[z \mapsto \text{Int}](b)$
$E \vdash z \notin \text{dom}(E) : \text{Int}$	$E \vdash 0 : \text{Int}$	$b := z \leq 3 : \text{ok}$	E
$E[z \mapsto \text{Int}] \vdash z \leq 3 : \text{Bool}$	$z \notin \text{dom}(E)$	$z \in \text{dom}(E)$	
$E[z \mapsto \text{Int}] \vdash z : \text{Int}$	$E(b) = \text{Bool}$	$E[z \mapsto \text{Int}] \vdash 3 : \text{Int}$	

[ass_{TS}]

[comp_{TS}]

[dec_{TS}]

[num_{TS}]

[var_{TS}]

[leq_{TS}]

_____ [num_{TS}] _____

_____ [va] _____

_____ [le] _____

😊 Your answer is correct.

Solution:

		$E[z \mapsto \text{Int}](z) = \text{Int}$	
	[var _{TS}]	_____	[num _{TS}]
		$E[z \mapsto \text{Int}] \vdash z : \text{Int}$	$E[z \mapsto \text{Int}] \vdash 3 : \text{Int}$
		[leq _{TS}]	_____
		$E[z \mapsto \text{Int}](b) = \text{Bool}$	$E[z \mapsto \text{Int}] \vdash z \leq 3 : \text{Bool}$
$z \notin \text{dom}(E)$	[num _{TS}]	_____	[ass _{TS}]
		$E \vdash 0 : \text{Int}$	_____
		$E[z \mapsto \text{Int}] \vdash b := z \leq 3 : \text{ok}$	
[dec _{TS}]	_____		
	$E \vdash \text{var Int } z := 0; b := z \leq 3; \text{ok}$		

Question 15

Flag question

Correct

Mark 7.00 out of 7.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using **static variables & dynamic procedures** scope rules

$$\begin{array}{c}
 \frac{\sigma_3 \circ \mathcal{E}_2 \vdash x + \underline{1} \rightarrow_E 4}{[\text{ass}_{\text{BS}}] \quad \mathcal{E}_2, \pi_1 \vdash_{l_3} \langle S, \sigma_3 \rangle \rightarrow \sigma_4 \quad \pi_1(p) = ?} \\
 \frac{\sigma_2 \circ \mathcal{E}_2 \vdash \underline{5} \rightarrow_E 5 \quad \mathcal{E}_3, \pi_1 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4}{[\text{call}_{\text{BS}}]} \\
 \frac{[\text{dec}_{\text{BS}}] \quad \mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{var Int } x := \underline{5}; \text{call } p, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{block}_{\text{BS}}] \quad \mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \frac{[\text{proc}_{\text{BS}}] \quad \mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{dec}_{\text{BS}}] \quad \sigma_1 \circ \mathcal{E}_1 \vdash \underline{3} \rightarrow_E 3 \quad \mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var Int } x := \underline{3}; \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4
 \end{array}$$

where $S = y := x + \underline{1}$ and $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the window of your browser for a better view of the derivation).

Provide the definitions of the stores and environments listed below by selecting the appropriate options so that the above is a well-formed derivation (the value of π_1 that is omitted in the derivation should be specified below).

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$

$\sigma_3 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\sigma_4 = [l_0 \mapsto 4, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_0 = \emptyset$

$\pi_1 = [p \mapsto (S, \mathcal{E}_2)]$

😊 **Solution:**

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3]$

$\sigma_3 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5]$

$\sigma_4 = \sigma[l_0 \mapsto 4, l_1 \mapsto 3, l_2 \mapsto 5]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_1 = [p \mapsto (S, \mathcal{E}_2)]$

Question 16

Flag question

Correct

Mark 7.00 out of 7.00

Consider the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using **fully dynamic** scope rules

$$\begin{array}{c}
 \sigma_3 \circ \mathcal{E}_3 \vdash x + \underline{1} \rightarrow_E 6 \\
 \text{[assBS]} \frac{}{\mathcal{E}_3, \pi_1 \vdash_{l_3} \langle S, \sigma_3 \rangle \rightarrow \sigma_4} \quad \pi_1(p) = ? \\
 \text{[callBS]} \frac{}{\mathcal{E}_3, \pi_1 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4} \\
 \sigma_2 \circ \mathcal{E}_2 \vdash \underline{5} \rightarrow_E 5 \\
 \text{[decBS]} \frac{}{\mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{var Int } x := \underline{5}; \text{call } p, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \text{[blockBS]} \frac{}{\mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \text{[procBS]} \frac{}{\mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \sigma_1 \circ \mathcal{E}_1 \vdash \underline{3} \rightarrow_E 3 \\
 \text{[decBS]} \frac{}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var Int } x := \underline{3}; \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4}
 \end{array}$$

where $S = y := x + \underline{1}$ and $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the window of your browser for a better view of the derivation).

Provide the definitions of the stores and environments listed below by selecting the appropriate options so that the above is a well-formed derivation (the value of π_1 that is omitted in the derivation should be specified below).

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$

$\sigma_3 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\sigma_4 = [l_0 \mapsto 6, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_0 = \emptyset$

$\pi_1 = [p \mapsto S]$

😊 **Solution:**

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3]$

$\sigma_3 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5]$

$\sigma_4 = \sigma[l_0 \mapsto 6, l_1 \mapsto 3, l_2 \mapsto 5]$

$\mathcal{E}_1 = [y \mapsto 0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_0 = \emptyset$

$\pi_1 = [p \mapsto S]$

Question 17

Flag question

Correct

Mark 15.00 out of 15.00

Consider the typed Bip statement below:

```
var Int x := 5;
var Int y := 2;
proc p is (y := x + 1; x := y - 1);
proc q is call p;
begin
  var Int x := 3;
  proc p is x := x + 3;
  call q
end
```

What is the final value assigned to the (global) variable y under the following scope rules?

- *Fully static scope rules* (static variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rules* (dynamic variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rules* (static variable bindings and dynamic procedure name bindings): $y =$ ✓
- *Fully dynamic scope rules* (dynamic variable bindings and dynamic procedure name bindings): $y =$ ✓

 Solution:

- *Fully static scope rules* (static variable bindings and static procedure name bindings): $y = 6$
- *Mixed scope rules* (dynamic variable bindings and static procedure name bindings): $y = 4$
- *Mixed scope rules* (static variable bindings and dynamic procedure name bindings): $y = 2$
- *Fully dynamic scope rules* (dynamic variable bindings and dynamic procedure name bindings): $y = 2$

Question 18

Flag question

Correct

Mark 6.00 out of 6.00

Consider the typed Bump statement below

```
var Int x := 1;
var Int y := 0;
proc p(strgy Int x) is (y := x + 3; x := 2);
call p(y);
x := x + y
```

where the call to the procedure p is declared with a generic parameter passing strategy $\text{strgy} \in \{\text{val}, \text{ref}, \text{name}\}$

What is the final value assigned to the (global) variable x under the following parameter passing strategies? (Assume fully static scope rules)

- **Call-by-value** (i.e, $\text{strgy} = \text{val}$): $x =$ ✓
- **Call-by-reference** (i.e, $\text{strgy} = \text{ref}$): $x =$ ✓
- **Call-by-name** (i.e, $\text{strgy} = \text{name}$): $x =$ ✓

 Solution:

- **Call-by-value** (i.e, $\text{strgy} = \text{val}$): $x = 4$
- **Call-by-reference** (i.e, $\text{strgy} = \text{ref}$): $x = 3$
- **Call-by-name** (i.e, $\text{strgy} = \text{name}$): $x = 5$

Finish review



Status
Finished

Started
Monday, 16 June 2025, 1:16 PM

Completed
Monday, 16 June 2025, 1:29 PM

Duration
12 mins 48 secs

Grade
99.59 out of 130.00 (76.61%)

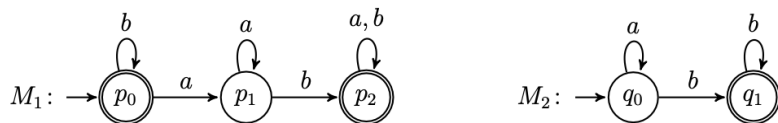
Question 1

Flag question

Correct

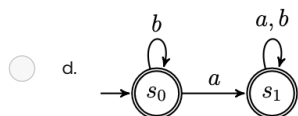
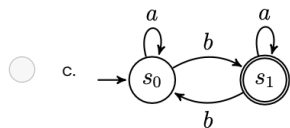
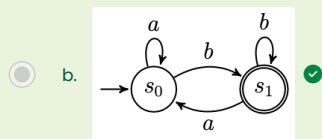
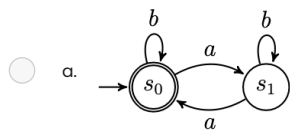
Mark 5.00 out of 5.00

Let M_1 and M_2 be the finite automata below:



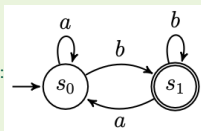
Which of the following automata recognise the language $L(M_1) \circ L(M_2)$ (the concatenation of the languages recognised by M_1 and M_2)?

Select one:



😊 Your answer is correct.

The correct answer is:



Question 2

Flag question

Not answered

Marked out of 10.00

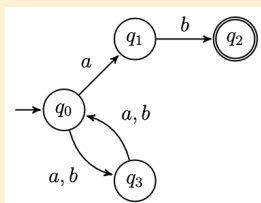
"Code-draw" the state diagram of a **nondeterministic finite automaton with at most 6 states** that recognises the language

$$L = \{w \in \{a, b\}^* \mid w \text{ ends in } ab \text{ and } |w| \text{ is even}\}.$$

Observe that any solution with less than 6 states is also accepted.

The syntax of reference to code-draw the automaton is that of the [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look at how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

 **Solution:**


#states

q0

q1

q2

q3

#initial

q0

#accepting

q2

#alphabet

a

b

#transitions

q0:a>q1

q0:a>q3

q0:b>q3

q1:b>q2

q3:a>q0

q3:b>q0

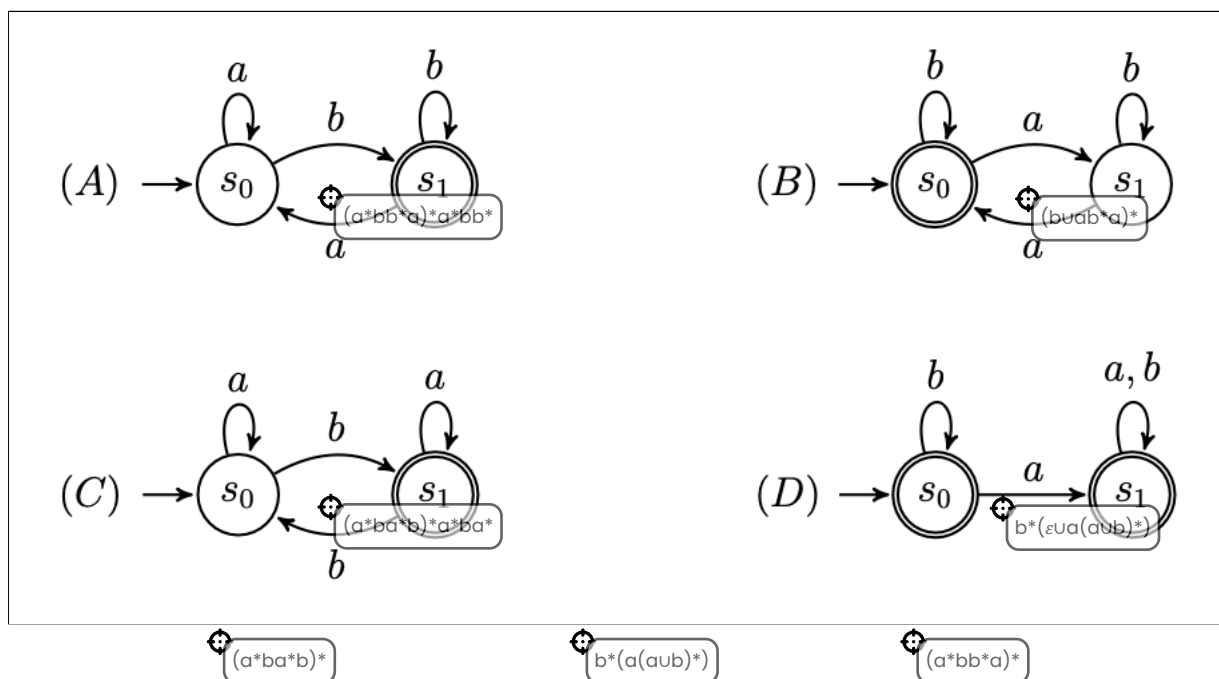
Question 3

Flag question

Correct

Mark 5.00 out of 5.00

Match the automata with the corresponding equivalent regular expressions .



😊 Your answer is correct.

Solution:A - $(a^*bb^*a)^*a^*bb^*$;B - $(b \cup ab^*a)^*$;C - $(a^*ba^*b)^*a^*ba^*$;D - $b^*(\varepsilon \cup a(a \cup b)^*)$.

Question 4

Flag question

Not answered

Marked out of 15.00

By using the Pumping Lemma, prove that the language $L = \{wb^n \mid w \in \{a, b\}^* \text{ and } n = |w|\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking the symbol $\equiv$ to expand the editor console, and then $\times$ to select the equation editor).

Solution:

To show that L is nonregular we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to choose $w = a^p b^p$. (Note that $w \in L$ and $|w| = 2p \geq p$. So we are playing according to the rules of the game).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$, and $|xy| \leq p$
4. For any decomposition of w into xyz as above, we show that for $i = 0$, $xy^i z \notin L$.
Observe that by the condition $|xy| \leq p$ we know that xy must occur within a^p and so $y = a^k$ for some $k > 0$. Thus, $xy^0 z = a^{p-k} b^p \notin L$, because $p - k \neq p$.

Question 5

Flag question

Correct

Mark 5.00 out of 5.00

Select the *most precise* option for each of the following languages:

- $\{ww \mid w \in \{a, b\}^* \text{ starts and ends with a different symbol}\}$: Not context-free ∇ ✓
- $\{a^n w b^n \mid n \geq 0 \text{ and } w \in \{a, b\}^*\}$: Regular ∇ ✓
- $\{b^n b^n \mid n > 0\}$: Regular ∇ ✓
- $\{a^n b a^{2n} \mid n \geq 0\}$: Context-free ∇ ✓
- $\{(ab)^n b^m a^n \mid n, m \geq 0\}$: Context-free ∇ ✓
- $\{(ab)^n b^n a^n \mid n \geq 0\}$: Not context-free ∇ ✓

Solution:

- $\{ww \mid w \in \{a, b\}^* \text{ starts and ends with a different symbol}\}$: Not context-free (It can be proven using the Pumping lemma for context-free languages. The proof is similar to Example 2.38 (pg. 129) in Michael Sipser's book *Introduction to the Theory of Computation*).
- $\{a^n w b^n \mid n \geq 0 \text{ and } w \in \{a, b\}^*\}$: Regular. (It is equal to language a, b^* . One containment is obvious, the other follows by picking $n = 0$).
- $\{b^n b^n \mid n > 0\}$: Regular (It is equivalent to the regular expression $(bb)^*$).
- $\{a^n b a^{2n} \mid n \geq 0\}$: Context-free (It is generated by the grammar $S \rightarrow b \mid aSa$).
- $\{(ab)^n b^m a^n \mid n, m \geq 0\}$: Context-free (It is generated by the grammar $S \rightarrow B \mid abSa \quad B \rightarrow \varepsilon \mid bB$)
- $\{(ab)^n b^n a^n \mid n \geq 0\}$: Not context-free (It can be proven using the Pumping lemma for context-free languages. The proof is similar to Example 2.36 (pg. 128) in Michael Sipser's book *Introduction to the Theory of Computation*)

Question 6

Flag question

Not answered

Marked out of 5.00

Drag the languages next to the context-free grammar that generates them.

$S \rightarrow \varepsilon \mid aSb \mid aS$

$S \rightarrow \varepsilon \mid F$
 $F \rightarrow b \mid bF$

$S \rightarrow aSb \mid bSa$

$S \rightarrow \varepsilon \mid aSb \mid bSa \mid SS$

$S \rightarrow \varepsilon \mid aSb \mid bS$

Drag answer here

Drag answer here

Drag answer here

Drag answer here

Drag answer here

$\{a^n b^n \mid n \geq 0\} \cup \{b^n a^n \mid n \geq 0\}$

$\{a^n b^m \mid n \geq m \geq 0\}$

$\{wb^n \mid w \text{ has } n \text{ occurrences of } a\}$

$\{w \mid w \text{ has equally many } a\text{'s and } b\text{'s}\}$

$\{b^n \mid n \geq 0\}$

$\emptyset$

$\{w \mid w \text{ starts and ends with different symbol}\}$

$\{\varepsilon\} \cup \{b^n b^n \mid n > 0\}$

Your answer is incorrect.
The correct answer is:

$S \rightarrow \varepsilon \mid aSb \mid aS$	$\{a^n b^m \mid n \geq m \geq 0\}$
$S \rightarrow \varepsilon \mid F$ $F \rightarrow b \mid bF$	$\{b^n \mid n \geq 0\}$
$S \rightarrow aSb \mid bSa$	$\emptyset$
$S \rightarrow \varepsilon \mid aSb \mid bSa \mid SS$	$\{w \mid w \text{ has equally many } a\text{'s and } b\text{'s}\}$
$S \rightarrow \varepsilon \mid aSb \mid bS$	$\{wb^n \mid w \text{ has } n \text{ occurrences of } a\}$

Question 7

Flag question

Correct

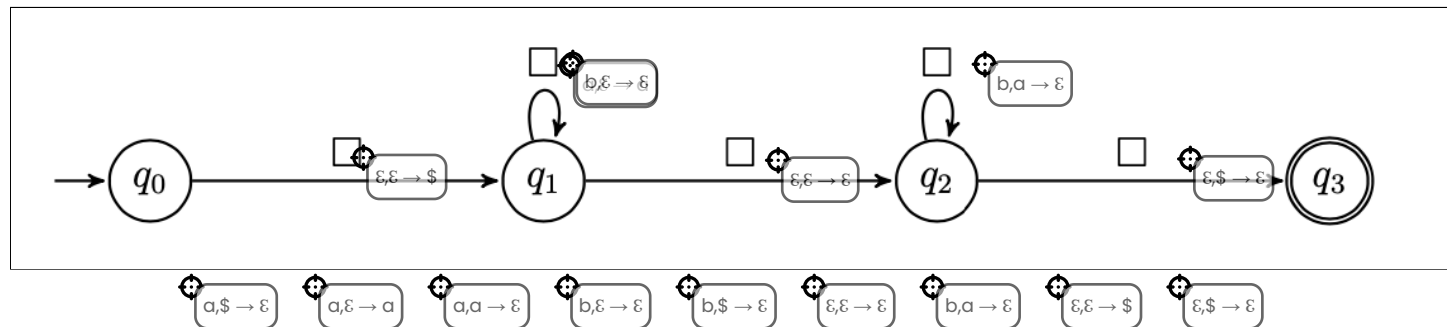
Mark 10.00 out of 10.00

Fill in the missing labels in the push-down automaton below so that it generates the language

$$L = \{wb^n \mid w \text{ has } n \text{ occurrences of } a\}.$$

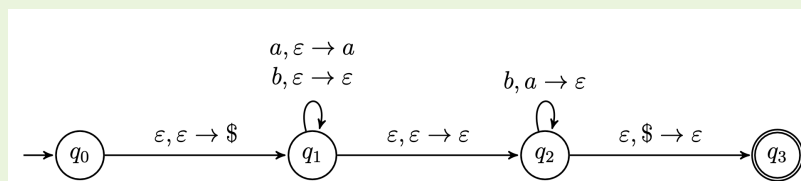
Instructions:

- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



😊 Your answer is correct.

Solution:



Question 8

Flag question

Correct

Mark 5.00 out of 5.00

Complete the context-free grammar below so that it generates the language

$$L = \{a^n w \mid w \in \{a, b\}^* \text{ has } n \text{ occurrences of } a\}$$

$S \rightarrow$ ☒ | ☒ | ☒ | ☒ | ☒ b

😊 Solution:

$$S \rightarrow \varepsilon \mid aSa \mid Sb$$

Question 9

Flag question

Correct

Mark 10.00 out of 10.00

Match each pushdown automaton to the language it recognises.

Instructions:

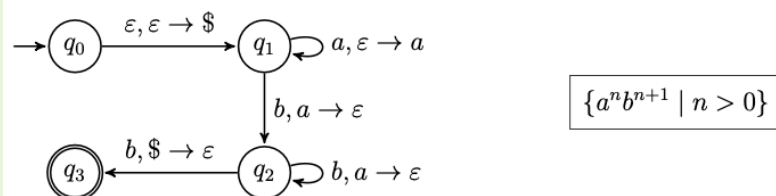
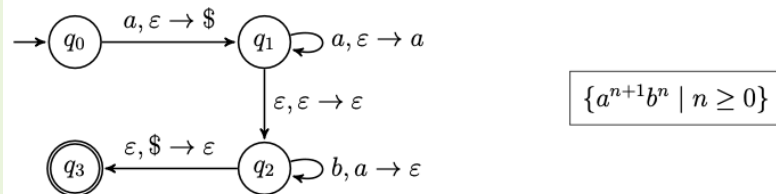
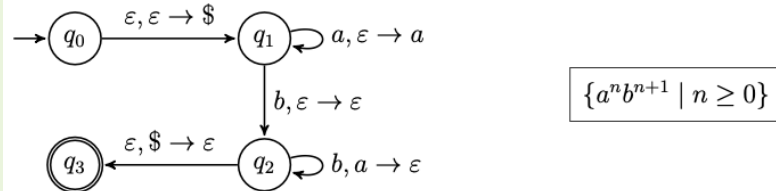
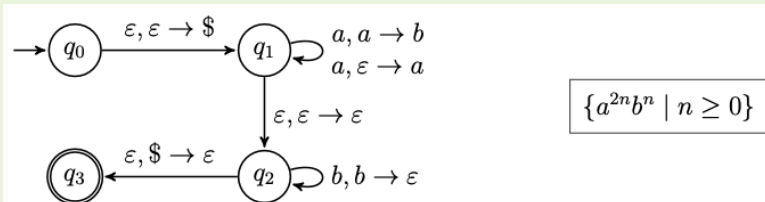
- Scroll down to see all the available options.
- Drag and drop the items into the boxes.
- The items can be used zero or more times.

	$\{a^{2n}b^n \mid n \geq 0\}$
	$\{a^n b^{n+1} \mid n \geq 0\}$
	$\{a^{n+1}b^n \mid n \geq 0\}$
	$\{a^n b^{n+1} \mid n > 0\}$

- | | | | | |
|--------------------------------|-----------------------------|------------------------------|--------------------------|---------------------------------|
| $\{a^{n+1}b^n \mid n \geq 0\}$ | $\{a^{n+1}b^n \mid n > 0\}$ | $\{a^n b^n \mid n \geq 0\}$ | $\{a^n b^n \mid n > 0\}$ | $\{a^n b^{n+1} \mid n \geq 0\}$ |
| $\{a^{2n}b^n \mid n \geq 0\}$ | | $\{a^n b^{n+1} \mid n > 0\}$ | | |

😊 Your answer is correct.

Solution:



Question 10

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree for the evaluation of $(x \cdot x) + 2y$ according to the big-step operational semantics for typed Bims expressions, assuming that $s = [x \rightarrow 2, y \rightarrow 3]$.

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$$\frac{[var_{BS}] \quad s(x) = 2}{s \vdash x \rightarrow_E 2}$$

$$\frac{[mult_{BS}] \quad s \vdash x \rightarrow_E 2 \quad s \vdash x \rightarrow_E 2}{s \vdash x \cdot x \rightarrow_E 4}$$

$$\frac{[par_{BS}] \quad s \vdash x \cdot x \rightarrow_E 4}{s \vdash (x \cdot x) \rightarrow_E 4}$$

$$\frac{[sum_{BS}] \quad s \vdash (x \cdot x) \rightarrow_E 4 \quad s \vdash 2y \rightarrow_E 6}{s \vdash (x \cdot x) + 2y \rightarrow_E 10}$$

$$\frac{[var_{BS}] \quad s(x) = 2}{s \vdash x \rightarrow_E 2}$$

$$\frac{[num_{BS}] \quad 2 \rightarrow_{Int} 2}{s \vdash 2 \rightarrow_E 2}$$

$$\frac{[var_{BS}] \quad s(y) = 3}{s \vdash y \rightarrow_E 3}$$

$s \vdash 2y \rightarrow_E 6$

$s(x) = 2$

$2 \rightarrow_{Int} 2$

$s \vdash x \rightarrow_E 2$

$s \vdash x \rightarrow_E 3$

$s(x) = 3$

$s(y) = 3$

$s \vdash y \rightarrow_E 3$

$s \vdash x \cdot x \rightarrow_E 4$

$s \vdash 2 \rightarrow_E 2$

$s \vdash (x \cdot x) \rightarrow_E 4$

[mult_{BS}]

[var_{BS}]

[sum_{BS}]

[par_{BS}]

😊 Your answer is correct.

Solution:

$$\begin{array}{c}
 \frac{[var_{BS}] \quad s(x) = 2}{s \vdash x \rightarrow_E 2} \quad \frac{[var_{BS}] \quad s(x) = 2}{s \vdash x \rightarrow_E 2} \quad \frac{[num_{BS}] \quad 2 \rightarrow_{Int} 2}{s \vdash 2 \rightarrow_E 2} \quad \frac{[var_{BS}] \quad s(y) = 3}{s \vdash y \rightarrow_E 3} \\
 \frac{[mult_{BS}] \quad s \vdash x \rightarrow_E 2 \quad s \vdash x \rightarrow_E 2}{s \vdash x \cdot x \rightarrow_E 4} \quad \frac{[par_{BS}] \quad s \vdash x \cdot x \rightarrow_E 4}{s \vdash (x \cdot x) \rightarrow_E 4} \quad \frac{[mult_{BS}] \quad s \vdash 2 \rightarrow_E 2 \quad s \vdash y \rightarrow_E 3}{s \vdash 2y \rightarrow_E 6} \\
 \frac{[sum_{BS}] \quad s \vdash (x \cdot x) \rightarrow_E 4 \quad s \vdash 2y \rightarrow_E 6}{s \vdash (x \cdot x) + 2y \rightarrow_E 10}
 \end{array}$$

Question 11

Flag question

Correct

Mark 5.00 out of 5.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the *state model* big-step operational semantics for typed Bims statements, where

 $W = \text{while } y < 3 \text{ do } y := y + \underline{2},$

and

 $s_1 = s_0[y \mapsto 1],$
 $s_2 = s_0[y \mapsto 3].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

[assBS]

$s_1 \vdash y + \underline{2} \rightarrow_E 3$

s_2

[assBS]

$\langle y := y + \underline{2}, s_1 \rangle \rightarrow s_2$

$\langle 1$

[assBS]

$s_0 \vdash \underline{1} \rightarrow_E 1$

$s_0 \rangle \rightarrow s_2$

[while-ttBS]

$\langle y := y + \underline{2}; W, s$

[compBS]

$\langle y := \underline{1}, s_0 \rangle \rightarrow s_1$

$\langle y := y + \underline{2}, s_1 \rangle \rightarrow s_2$

$\langle y := \underline{1}, s_0 \rangle \rightarrow s_1$

$s_0 \vdash \underline{1} \rightarrow_E 1$

[compBS]

$\langle y := y + \underline{2}, s_1 \rangle \rightarrow s_2$

$s_1 \vdash y + \underline{2} \rightarrow_E 3$

$\langle y := \underline{1}, s_0 \rangle \rightarrow s_1$

$\langle W, s_1 \rangle \rightarrow s_2$

$s_1 \vdash y < 3 \rightarrow_E tt$

$s_2 \vdash y < 3 \rightarrow_E ff$

[compBS]

[while-ttBS]

[assBS]

[while-ffBS]

😊 Your answer is correct.

Solution:

[assBS]

$s_1 \vdash y + \underline{2} \rightarrow_E 3$

[while-ffBS]

$s_2 \vdash y < 3 \rightarrow_E ff$

[compBS]

$\langle y := y + \underline{2}, s_1 \rangle \rightarrow s_2$

$\langle W, s_2 \rangle \rightarrow s_2$

[assBS]

$s_0 \vdash \underline{1} \rightarrow_E 1$

[while-ttBS]

$\langle y := y + \underline{2}; W, s_1 \rangle \rightarrow s_2$

$s_1 \vdash y < 3 \rightarrow_E tt$

[compBS]

$\langle y := \underline{1}, s_0 \rangle \rightarrow s_1$

$\langle W, s_1 \rangle \rightarrow s_2$

$\langle y := \underline{1}; W, s_0 \rangle \rightarrow s_2$

Question 12

Flag question

Correct

Mark 5.00 out of 5.00

Consider the following derivation tree constructed according to the rules of the state model big-step operational semantics for typed Bims statements. (Enlarge the window for a better view of the derivation).

$$\begin{array}{c}
 \text{[ass}_{\text{BS}}\text{]} \frac{s_0 \vdash 3y \rightarrow_E 6}{\langle y := 3y, s_0 \rangle \rightarrow s_1} \quad \text{[comp}_{\text{BS}}\text{]} \frac{\begin{array}{c} \text{[ass}_{\text{BS}}\text{]} \frac{s_1 \vdash y + \underline{1} \rightarrow_E 7}{\langle x := y + \underline{1}, s_1 \rangle \rightarrow s_2} \quad \text{[ass}_{\text{BS}}\text{]} \frac{s_2 \vdash x - 2y \rightarrow_E -5}{\langle x := x - 2y, s_2 \rangle \rightarrow s_3} \\ \langle x := y + \underline{1}; x := x - 2y, s_1 \rangle \rightarrow s_3 \end{array}}{\langle y := 3y; x := y + \underline{1}; x := x - 2y, s_0 \rangle \rightarrow s_3}
 \end{array}$$

Fill in the blanks in the definitions below so that the above is a well-formed derivation; use "-" if the value is undefined.

$$s_0 = [y \mapsto 2]$$

$$s_1 = [x \mapsto \text{ }, y \mapsto \text{ }]$$

$$s_2 = [x \mapsto \text{ }, y \mapsto \text{ }]$$

$$s_3 = [x \mapsto \text{ }, y \mapsto \text{ }]$$

**Solution:**

$$s_1 = s[y \mapsto 6] \text{ (} x \text{ undefined)}$$

$$s_2 = s[x \mapsto 7, y \mapsto 6]$$

$$s_3 = s[x \mapsto -5, y \mapsto 6]$$

Question 13

Flag question

Correct

Mark 5.00 out of 5.00

Let $W = \text{while } x \leq \underline{6} \text{ do } x := x + \underline{4}; y := x - \underline{2}$.

Fill in the blanks in the derivation sequence below according to the rules of the state model *small-step* semantics for typed Bims statements. (Expand the window of your browsers for better visualisation).

$\langle W, [x \mapsto 0, y \mapsto 0] \rangle$
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 0, y \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + \underline{4}; y := x - \underline{2}; W, [x \mapsto 0, y \mapsto 0] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := x - \underline{2}; W, [x \mapsto 4, y \mapsto 0] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 2] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 4, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + \underline{4}; y := x - \underline{2}; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := x - \underline{2}; W, [x \mapsto 8, y \mapsto 2] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 8, y \mapsto 6] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 8, y \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 8, y \mapsto 6] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 8, y \mapsto 6]$ (skip_{SS})

 Solution:

$\langle W, [x \mapsto 0, y \mapsto 0] \rangle$
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 0, y \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + \underline{4}; y := x - \underline{2}; W, [x \mapsto 0, y \mapsto 0] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := x - \underline{2}; W, [x \mapsto 4, y \mapsto 0] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 4, y \mapsto 2] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 4, y \mapsto 2] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + \underline{4}; y := x - \underline{2}; W, [x \mapsto 4, y \mapsto 2] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle y := x - \underline{2}; W, [x \mapsto 8, y \mapsto 2] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle W, [x \mapsto 8, y \mapsto 6] \rangle$ (ass_{SS}+comp-1_{SS})
 $\Rightarrow \langle \text{if } x \leq \underline{6} \text{ then } x := x + \underline{4}; y := x - \underline{2}; W \text{ else skip}, [x \mapsto 8, y \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 8, y \mapsto 6] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 8, y \mapsto 6]$ (skip_{SS})

Question 14

Flag question

Correct

Mark 3.00 out of 3.00

The derivation tree below, constructed according to the rules of the type system for Bims statements with variable declarations, contains **3 typos**.

Mark each typo with an "error" label by placing the target cursor (the top left circle) as close as possible to where you believe the error is located. You may use a maximum of 3 error labels.

Enlarge the window to see better the details of the derivation.

$$\begin{array}{c}
 \begin{array}{c}
 \begin{array}{c}
 \frac{E(x) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E \vdash x : \text{Int}}} \quad \frac{E(y) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E \vdash y : \text{Int}}} \quad \frac{E'(c) = \text{Bool}}{[\text{var}_{\text{TS}}] \frac{}{E' \vdash c : \text{Bool}}} \quad \frac{E'(x) = \text{Int} \quad \frac{E'(y) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E' \vdash y : \text{Int}}}}{[\text{ass}_{\text{TS}}] \frac{}{E' \vdash x := y : \text{Int}}} \quad \frac{}{[\text{skip}_{\text{TS}}] \frac{}{E' \vdash \text{skip} : \text{ok}}} \\
 \frac{c \in \text{dom}(E)}{[\text{leq}_{\text{TS}}] \frac{}{E \vdash x \leq y : \text{Int}}} \quad \frac{E' \vdash c : \text{Bool} \quad E' \vdash x := y : \text{Int} \quad E' \vdash \text{skip} : \text{ok}}{[\text{if}_{\text{TS}}] \frac{}{E' \vdash \text{if } c \text{ then } x := y \text{ else skip} : \text{ok}}} \\
 \frac{}{[\text{dec}_{\text{TS}}] \frac{}{E \vdash \text{var Bool } c := x \leq y; \text{if } c \text{ then } x := y \text{ else skip} : \text{ok}}}
 \end{array}
 \end{array}$$

where $E = [x \mapsto \text{Int}, y \mapsto \text{Int}]$ and $E' = [x \mapsto \text{Int}, y \mapsto \text{Int}, c \mapsto \text{Bool}]$.



😊 Your answer is correct.

Solution:

$$\begin{array}{c}
 \begin{array}{c}
 \begin{array}{c}
 \frac{E(x) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E \vdash x : \text{Int}}} \quad \frac{E(y) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E \vdash y : \text{Int}}} \quad \frac{E'(c) = \text{Bool}}{[\text{var}_{\text{TS}}] \frac{}{E' \vdash c : \text{Bool}}} \quad \frac{E'(x) = \text{Int} \quad \frac{E'(y) = \text{Int}}{[\text{var}_{\text{TS}}] \frac{}{E' \vdash y : \text{Int}}}}{[\text{ass}_{\text{TS}}] \frac{}{E' \vdash x := y : \text{ok}}} \quad \frac{}{[\text{skip}_{\text{TS}}] \frac{}{E' \vdash \text{skip} : \text{ok}}} \\
 \frac{c \notin \text{dom}(E)}{[\text{leq}_{\text{TS}}] \frac{}{E \vdash x \leq y : \text{Bool}}} \quad \frac{E' \vdash c : \text{Bool} \quad E' \vdash x := y : \text{ok} \quad E' \vdash \text{skip} : \text{ok}}{[\text{if}_{\text{TS}}] \frac{}{E' \vdash \text{if } c \text{ then } x := y \text{ else skip} : \text{ok}}} \\
 \frac{}{[\text{dec}_{\text{TS}}] \frac{}{E \vdash \text{var Bool } c := x \leq y; \text{if } c \text{ then } x := y \text{ else skip} : \text{ok}}}
 \end{array}
 \end{array}$$

Question 15

Flag question

Correct

Mark 7.00 out of 7.00

Consider the following derivation tree* constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using **dynamic variables & static procedures** scope rules

$$\begin{array}{c}
 \frac{\sigma_3 \circ \mathcal{E}_3 \vdash x + \underline{1} \rightarrow_E 6}{[\text{ass}_{\text{BS}}]} \\
 \frac{\mathcal{E}_3, \Pi_1 \vdash_{l_3} \langle S, \sigma_3 \rangle \rightarrow \sigma_4 \quad \Pi_1(p) = ?}{[\text{call-1}_{\text{BS}}]} \\
 \frac{\mathcal{E}_3, \Pi_1 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4 \quad p \notin \text{dom}(\emptyset)}{[\text{call-2}_{\text{BS}}]} \\
 \frac{\sigma_2 \circ \mathcal{E}_2 \vdash \underline{5} \rightarrow_E 5 \quad \mathcal{E}_3, \Pi_2 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4}{[\text{dec}_{\text{BS}}]} \\
 \frac{\mathcal{E}_2, \Pi_2 \vdash_{l_2} \langle \text{var lnt } x := \underline{5}; \text{call } p, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{block}_{\text{BS}}]} \\
 \frac{\mathcal{E}_2, \Pi_1 \vdash_{l_2} \langle \text{begin var lnt } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{proc}_{\text{BS}}]} \\
 \frac{\sigma_1 \circ \mathcal{E}_1 \vdash \underline{3} \rightarrow_E 3 \quad \mathcal{E}_2, \Pi_0 \vdash_{l_2} \langle \text{proc } p \text{ is } S; \text{begin var lnt } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4}{[\text{dec}_{\text{BS}}]} \\
 \mathcal{E}_1, \Pi_0 \vdash_{l_1} \langle \text{var lnt } x := \underline{3}; \text{proc } p \text{ is } S; \text{begin var lnt } x := \underline{5}; \text{call } p \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4
 \end{array}$$

where $S = y := x + \underline{1}$ and $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the window of your browser for a better view of the derivation).

(*) The value of $\Pi_1(p)$ is omitted in the derivation because it is part of the solution that you need to provide below.

Provide the definitions of the stores and environments listed below by selecting the appropriate options so that the above is a well-formed derivation.

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$

$\sigma_3 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\sigma_4 = [l_0 \mapsto 6, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\Pi_0 = \emptyset$

$\Pi_1 = [p \mapsto S]$

$\Pi_2 = \emptyset[p \mapsto S]$

😊 **Solution:**

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3]$

$\sigma_3 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5]$

$\sigma_4 = \sigma[l_0 \mapsto 6, l_1 \mapsto 3, l_2 \mapsto 5]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\Pi_0 = \emptyset$

$\Pi_1 = [p \mapsto S]$

$\Pi_2 = \emptyset[p \mapsto S]$

Question 16

Flag question

Partially correct

Mark 6.59 out of 7.00

Consider the following derivation tree* constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip using **fully dynamic** scope rules

$$\begin{array}{c}
 \sigma_3 \circ \mathcal{E}_3 \vdash x + \underline{1} \rightarrow_E 6 \\
 \text{[ass}_{\text{BS}}] \frac{}{\mathcal{E}_3, \pi_1 \vdash_{l_3} \langle S, \sigma_3 \rangle \rightarrow \sigma_4 \quad \pi_1(p) = ?} \\
 \text{[call}_{\text{BS}}] \frac{\sigma_2 \circ \mathcal{E}_2 \vdash \underline{5} \rightarrow_E 5 \quad \mathcal{E}_3, \pi_1 \vdash_{l_3} \langle \text{call } p, \sigma_3 \rangle \rightarrow \sigma_4}{\mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{var Int } x := \underline{5}; \text{call } p, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \text{[block}_{\text{BS}}] \frac{}{\mathcal{E}_2, \pi_1 \vdash_{l_2} \langle \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4} \\
 \text{[proc}_{\text{BS}}] \frac{\sigma_1 \circ \mathcal{E}_1 \vdash \underline{3} \rightarrow_E 3 \quad \mathcal{E}_2, \pi_0 \vdash_{l_2} \langle \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_2 \rangle \rightarrow \sigma_4}{\mathcal{E}_1, \pi_0 \vdash_{l_1} \langle \text{var Int } x := \underline{3}; \text{proc } p \text{ is } S; \text{begin var Int } x := \underline{5}; \text{call } p \text{ end}, \sigma_1 \rangle \rightarrow \sigma_4} \\
 \text{[dec}_{\text{BS}}] \frac{}{}
 \end{array}$$

where $S = y := x + \underline{1}$ and $l_{i+1} = \text{nxt}(l_i)$. (Enlarge the window of your browser for a better view of the derivation).

(*) The value of $\pi_1(p)$ is omitted in the derivation because it is part of the solution that you need to provide below.

Provide the definitions of the stores and environments listed below by selecting the appropriate options so that the above is a well-formed derivation.

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto \text{undefined}, l_3 \mapsto \text{undefined}]$

$\sigma_3 = [l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\sigma_4 = [l_0 \mapsto 5, l_1 \mapsto 3, l_2 \mapsto 5, l_3 \mapsto \text{undefined}]$

$\mathcal{E}_1 = [y \mapsto l_0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_0 = \emptyset$

$\pi_1 = [p \mapsto S]$

😊 **Solution:**

$\sigma_1 = \sigma[l_0 \mapsto 0]$

$\sigma_2 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3]$

$\sigma_3 = \sigma[l_0 \mapsto 0, l_1 \mapsto 3, l_2 \mapsto 5]$

$\sigma_4 = \sigma[l_0 \mapsto 6, l_1 \mapsto 3, l_2 \mapsto 5]$

$\mathcal{E}_1 = [y \mapsto 0]$

$\mathcal{E}_2 = [x \mapsto l_1, y \mapsto l_0]$

$\mathcal{E}_3 = [x \mapsto l_2, y \mapsto l_0]$

$\pi_0 = \emptyset$

$\pi_1 = [p \mapsto S]$

Question 17

Flag question

Correct

Mark 15.00 out of 15.00

Consider the typed Bip statement below:

```
var Int x := 2;
var Int y := 3;
proc p is (y := x + 1; x := 3 · y);
proc q is (call p; y := x + 1);
begin
  var Int x := 3;
  proc p is y := x - 2;
  call q
end
```

What is the final value assigned to the (global) variable y under the following scope rules?

- *Fully static scope rules* (static variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rules* (dynamic variable bindings and static procedure name bindings): $y =$ ✓
- *Mixed scope rules* (static variable bindings and dynamic procedure name bindings): $y =$ ✓
- *Fully dynamic scope rules* (dynamic variable bindings and dynamic procedure name bindings): $y =$ ✓

 Solution:

- *Fully static scope rules* (static variable bindings and static procedure name bindings): $y = 10$
- *Mixed scope rules* (dynamic variable bindings and static procedure name bindings): $y = 13$
- *Mixed scope rules* (static variable bindings and dynamic procedure name bindings): $y = 3$
- *Fully dynamic scope rules* (dynamic variable bindings and dynamic procedure name bindings): $y = 4$

Question 18

Flag question

Correct

Mark 8.00 out of 8.00

Consider the typed Bump statement below

```
var Int x := 2;
var Int y := 3;
proc p(strgy Int x) is (x := x + 3; y := x + 1; x := x - 1);
call p(y);
x := x + y
```

where the call to the procedure p is decalred with a generic parameter passing strategy $\text{strgy} \in \{\text{val}, \text{ref}, \text{name}\}$

What is the final value assigned to the (global) variable x under the following parameter passing strategies? (Assume fully static scope rules)

- **Call-by-value** (i.e, $\text{strgy} = \text{val}$): $x =$ ✓
- **Call-by-reference** (i.e, $\text{strgy} = \text{ref}$): $x =$ ✓
- **Call-by-name** (i.e, $\text{strgy} = \text{name}$): $x =$ ✓

 Solution:

- **Call-by-value** (i.e, $\text{strgy} = \text{val}$): $x = 9$
- **Call-by-reference** (i.e, $\text{strgy} = \text{ref}$): $x = 8$
- **Call-by-name** (i.e, $\text{strgy} = \text{name}$): $x = 7$

Finish review



Status

Finished

Started

Sunday, 15 June 2025, 11:43 AM

Completed

Sunday, 15 June 2025, 11:50 AM

Duration

6 mins 57 secs

Grade

50 out of 90 (56%)

Feedback

You got more than half of the questions right, but you can still do better than that. With a bit more effort you can get top scores!

Question 1

Flag question

Correct

Mark 1 out of 1

Which among the following is the state diagram of the deterministic automaton M below:

$$M = (Q, \Sigma, \delta, s, F)$$

$$Q = \{q_0, q_1, q_2\}$$

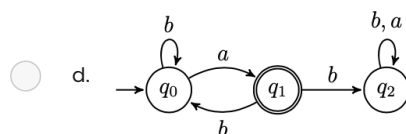
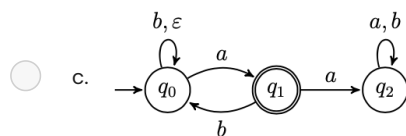
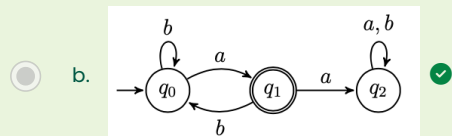
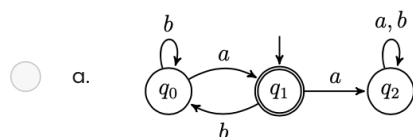
$$\Sigma = \{a, b\}$$

$$s = q_0$$

$$F = \{q_1\}$$

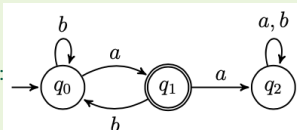
δ	a	b
q_0	q_1	q_0
q_1	q_2	q_0
q_2	q_2	q_2

Select one:



😊 Your answer is correct.

The correct answer is:



Question 2

[Flag question](#)

Not answered

Marked out of 4

"Code-draw" the state diagram of a **deterministic** finite automaton recognising the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has suffix } bb\}$$

The syntax of reference to code-draw the automaton is that of [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

😊 A correct solution is:

```
#states
q0
q1
q2
#initial
q0
#accepting
q2
#alphabet
a
b
#transitions
q0:a>q0
q0:b>q1
q1:a>q0
q1:b>q2
q2:a>q0
q2:b>q2
```

Note that the question requires the automaton to be **deterministic**. If determinism is specifically required, solutions using non-determinism won't be considered valid for the exam.

Question 3

[Flag question](#)

Not answered

Marked out of 4

"Code-draw" the state diagram of a finite automaton recognising the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has suffix } bb\}$$

The syntax of reference to code-draw the automaton is that of [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

😊 A correct solution is:

```
#states
q0
q1
q2
#initial
q0
#accepting
q2
#alphabet
a
b
#transitions
q0:a>q0
q0:b>q0
q0:b>q1
q1:b>q2
```

Note that the question does not require the automaton to be deterministic. If determinism is specifically required, solutions using non-determinism won't be considered valid for the exam.

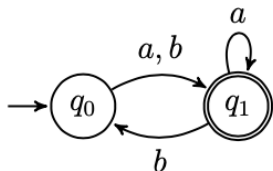
Question 4

Flag question

Correct

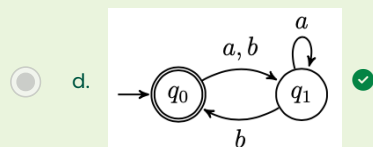
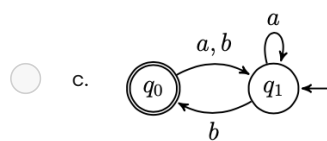
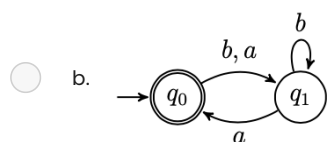
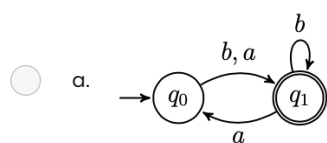
Mark 1 out of 1

Let M be the finite automaton below, recognising the language $L(M)$.



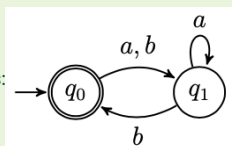
Which of the following automata recognises the language $\overline{L(M)}$ (the *complement* of $L(M)$)?

Select one:



😊 Your answer is correct.

The correct answer is:



Question 5

Flag question

Correct

Mark 1 out of 1

Which among the following is the state diagram of the nondeterministic automaton M below:

$$M = (Q, \Sigma, \delta, s, F)$$

$$Q = \{q_0, q_1, q_2\}$$

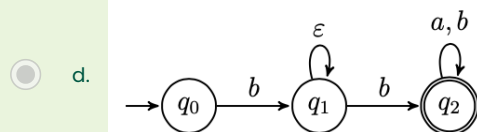
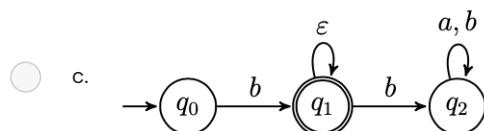
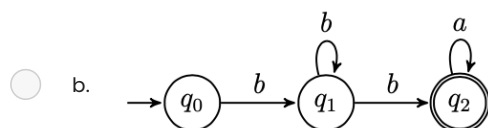
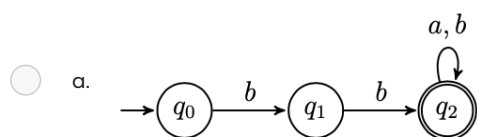
$$\Sigma = \{a, b\}$$

$$s = q_0$$

$$F = \{q_2\}$$

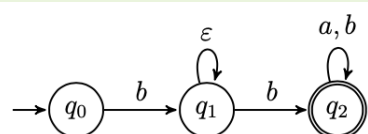
δ	a	b
q_0	$\emptyset$	$\{q_1\}$
q_1	$\emptyset$	$\{q_2\}$
q_2	$\{q_2\}$	$\emptyset$

Select one:



😊 Your answer is correct.

The correct answer is:



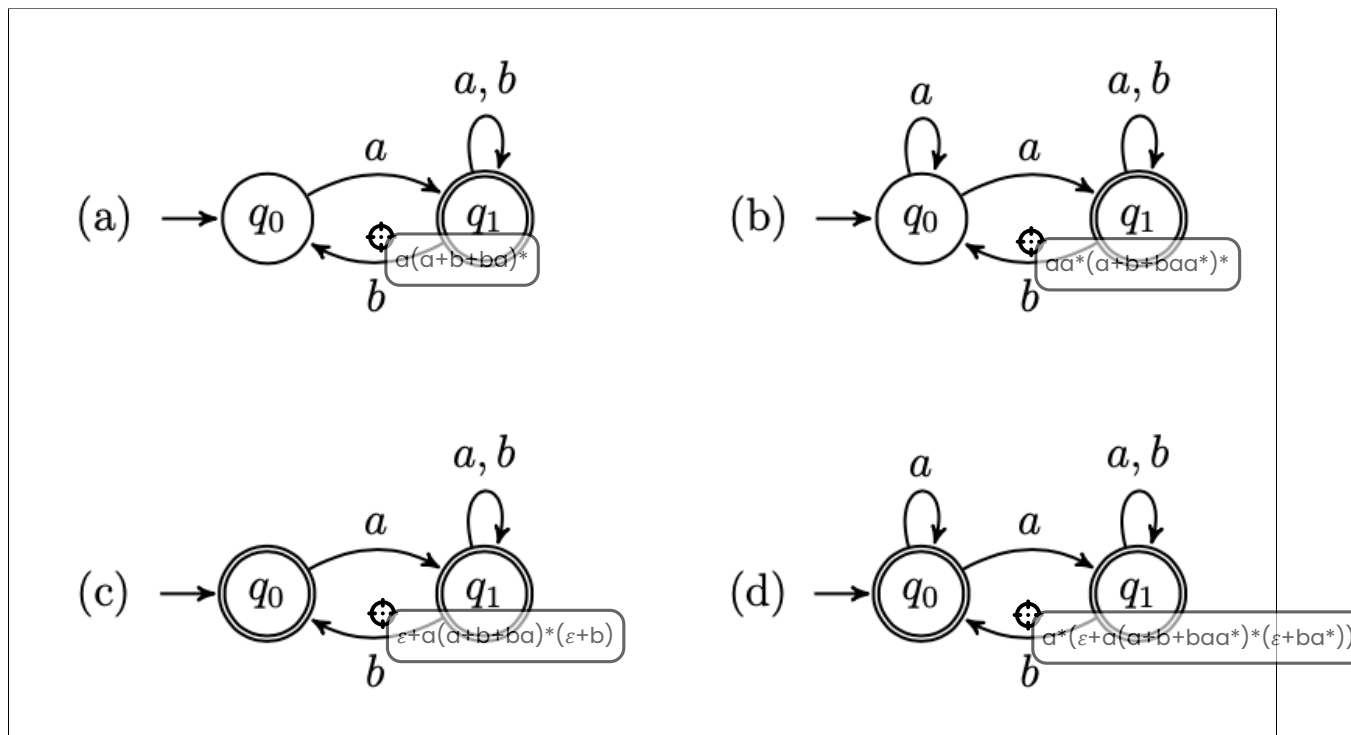
Question 6

Flag question

Correct

Mark 4 out of 4

Drag each regular expression onto the equivalent automaton.



😊 Your answer is correct.

Question 7

[Flag question](#)

Not answered

Marked out of 4

"Code-draw" the state diagram of a finite automaton recognising the language

$$L = \{w \in \{a, b\}^* \mid w \text{ has prefix } aa \text{ and suffix } ab\}$$

The syntax of reference to code-draw the automaton is that of [FSM simulator](#) (see "Input automaton" tab). To learn the syntax, generate a random automaton and look how it is "coded" in the language. Then click the "Create automaton" button to visualise it and simulate possible execution runs from a given input string.

RECOMMENDED: Before submitting your solution, **you are strongly encouraged to inspect, check, and test its correctness using all the facilities provided by the simulator**

😊 A correct solution is:

```
#states
q0
q1
q2
q3
q4
#initial
q0
#accepting
q4
#alphabet
a
b
#transitions
q0:a>q1
q1:a>q2
q2:b>q4
q2:$>q3
q3:a>q3
q3:b>q3
q3:$>q1
```

Note that the question does not require the automaton to be deterministic. If determinism is specifically required, solutions using non-determinism won't be considered valid for the exam.

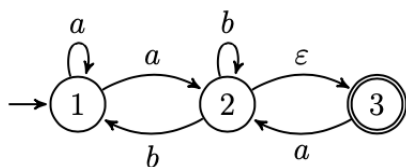
Question 8

Flag question

Correct

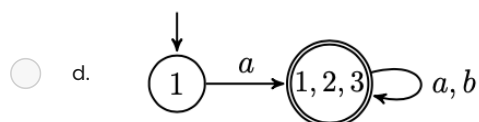
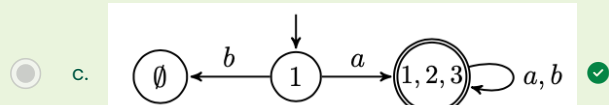
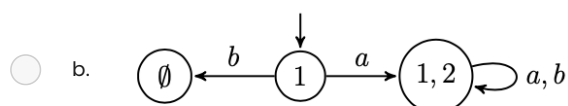
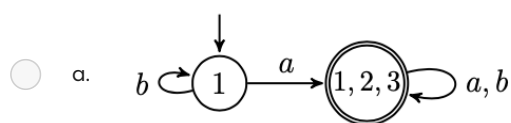
Mark 1 out of 1

Consider the NFA



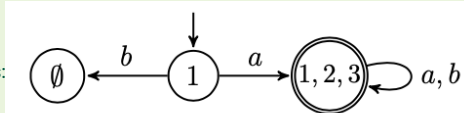
Which of the following automata is the determinization of the NFA above?

Select one:



😊 Your answer is correct.

The correct answer is:



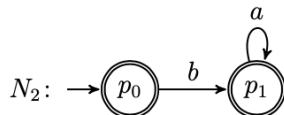
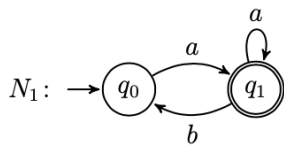
Question 9

Flag question

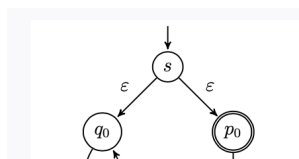
Not answered

Marked out of 1

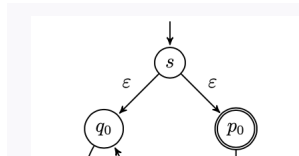
Consider the following NFAs, N_1 and N_2 , recognising the languages $L(N_1) = a(a \cup ba)^*$ and $L(N_2) = \varepsilon \cup ba^*$, respectively.



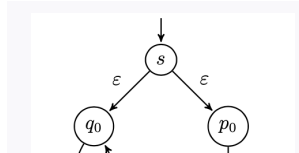
Match each of the languages to the right with an equivalent automaton to the left.



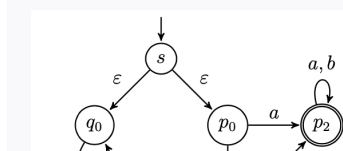
Drag answer here



Drag answer here



Drag answer here



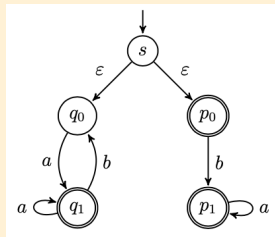
Drag answer here

 $L(N_1) \cup L(N_2)$ $L(N_1) \cap L(N_2)$ $L(N_1)^* \cup L(N_2)$ $L(N_1) \cup \overline{L(N_2)}$ 

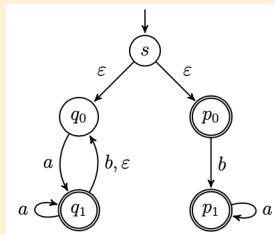
Your answer is incorrect.

Recall that finite automata are closed under all regular operations (i.e., union, concatenation and star operation) and complement. Moreover, as a consequence of DeMorgan's laws, they are also closed under intersections.

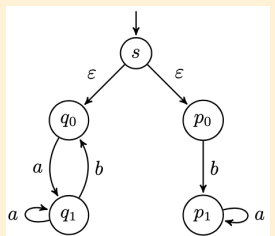
The correct answer is:



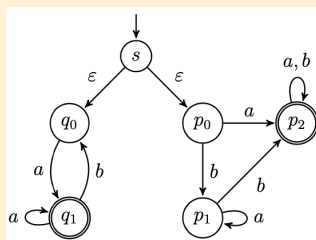
$$L(N_1) \cup L(N_2)$$



$$L(N_1)^* \cup L(N_2)$$



$$L(N_1) \cap L(N_2)$$



$$L(N_1) \cup \overline{L(N_2)}$$

Question 10

Flag question

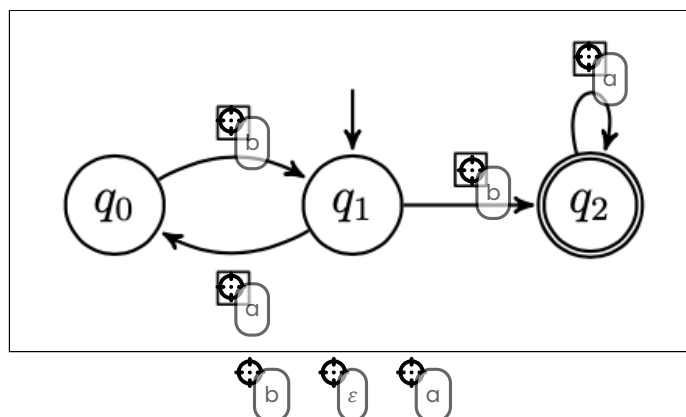
Correct

Mark 6 out of 6

Fill in the missing labels in the finite automaton below so that it becomes equivalent to the regular expression $(ab)^*ba^*$.

Instructions:

- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



😊 Your answer is correct.

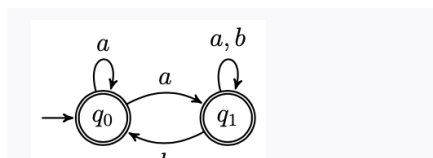
Question 11

Flag question

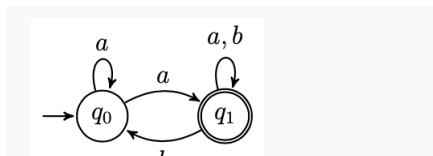
Not answered

Marked out of 8

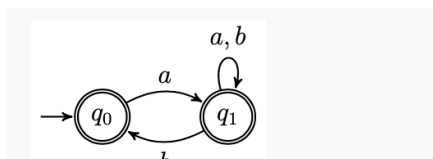
Match each regular expression to the right with an equivalent automaton to the left.



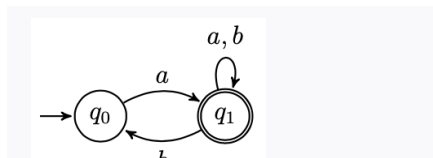
Drag answer here



Drag answer here



Drag answer here

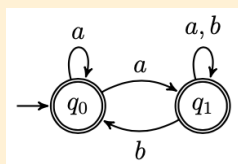
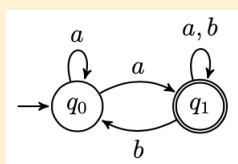
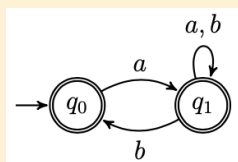
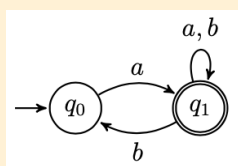


Drag answer here

 $aa^*(a \cup b \cup baa^*)^*$ $a(a \cup b \cup ba)^*$ $\varepsilon \cup a(a \cup b \cup ba)^*(\varepsilon \cup b)$ $a^*(\varepsilon \cup a(a \cup b \cup baa^*)^*(\varepsilon \cup ba^*))$

☹ Your answer is incorrect.

The correct answer is:

 $a^*(\varepsilon \cup a(a \cup b \cup baa^*)^*(\varepsilon \cup ba^*))$  $aa^*(a \cup b \cup baa^*)^*$  $\varepsilon \cup a(a \cup b \cup ba)^*(\varepsilon \cup b)$  $a(a \cup b \cup ba)^*$

Question 12

Flag question

Not answered

Marked out of 8

By using the Pumping Lemma, prove that the language $L = \{b^n a c^{n+1} \mid n \geq 0\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking  to expand the editor console, and then  to select the equation editor).

**Solution:**

To show that L is nonregular we use the game characterisation of the Pumping Lemma and provide a universal winning strategy against the demon.

1. The demon picks $p \geq 0$
2. A good response is to choose $w = b^p a c^{p+1}$. (Note that $w \in L$ and $|w| = 2(p+1) \geq p$. So we are playing according to the rules of the game).
3. The demon picks x, y, z such that $w = xyz$, $y \neq \varepsilon$, and $|xy| \leq p$
4. For any decomposition of w into xyz as above, we show that for $i = 0$, $xy^i z \notin L$.
Note that by the condition $|xy| \leq p$ we have that y must be consisting only of b 's. Assume $y = b^j$ for some $j > 0$. Then $xy^0 z = b^{p-j} a c^{p+1} \notin L$ because $p - j < p$.

Question 13

Flag question

Not answered

Marked out of 8

From the SS course (cf. *M. Sipser, Introduction to the Theory of Computation, Example 1.73, pp. 80*) we know that the language $\{0^n 1^n \mid n \geq 0\}$ is nonregular.

Use this fact to prove that $L = \{w \in \{0, 1\}^* \mid w \text{ has equal number of occurrences of 0's and 1's}\}$ is nonregular.

(Optional: you can write mathematical expressions using LaTeX syntax by clicking $\equiv$ to expand the editor console, and then $\times$ to select the equation editor).

 Solution:

We prove the nonregularity of L by using the fact that regular languages are closed under intersections. Assume, for the sake of finding a contradiction, that L is regular. Then, as regular languages are closed under intersection, the following

$$L \cap L(0^* 1^*) = \{0^n 1^n \mid n \geq 0\}$$

must also be regular. But this is a contradiction because we already know that $\{0^n 1^n \mid n \geq 0\}$ is nonregular. Thus, L must be nonregular.

Question 14

Flag question

Correct

Mark 1 out of 1

Tick the checkboxes corresponding to the languages that are *nonregular*.

☐ $\{a^n aa \mid n \geq 0\}$
☒ $\{ab^n c^m \mid 0 \leq n \leq m\}$  This language is nonregular. As an exercise, prove it by using the Pumping Lemma.

☐ $\{ab^n c \mid n \geq 0\}$
☐ $\{a^n aa^n \mid n \geq 0\}$
 Your answer is correct.

The correct answer is: $\{ab^n c^m \mid 0 \leq n \leq m\}$

Question 15

Flag question

Correct

Mark 4 out of 4

Match each language to the right with the context-free grammar to the left that generates it.

$S \rightarrow AB$ $A \rightarrow aA \mid \varepsilon$ $B \rightarrow bBb \mid \varepsilon$	$\{a^n b^{2n} \mid n \geq 0\}$	$\{a^{2n} b^{m+1} \mid n, m \geq 0\}$
$S \rightarrow \Sigma a \Sigma a \Sigma$ $\Sigma \rightarrow a \Sigma \mid b \Sigma \mid \varepsilon$	$\{w \in \{a, b\}^* \mid w \text{ has at least 2 } a\}$	$\{w \in \{a, b\}^* \mid w \text{ has at least 2 } a\}$
$S \rightarrow AB$ $A \rightarrow aAa \mid \varepsilon$ $B \rightarrow bB \mid b$	$\{a^{2n} b^{m+1} \mid n, m \geq 0\}$	$\{a^* b^{2n} \mid n \geq 0\}$
$S \rightarrow aaSb \mid \varepsilon$	$\{a^{2n} b^n \mid n \geq 0\}$	$\{a^{2n} b^n \mid n \geq 0\}$

😊 Your answer is correct.

Question 16

Flag question

Correct

Mark 2 out of 2

Tick all the checkboxes corresponding to the language generated by the following context-free grammar

 $S \rightarrow a \mid b \mid a \Sigma a \mid b \Sigma b$
 $\Sigma \rightarrow S \mid a \Sigma b \mid b \Sigma a$

- ☐ $\{w \mid w \text{ starts and ends with the same symbol}\}$
☒ $a(a \cup b)((a \cup b)(a \cup b))^* a \cup b(a \cup b)((a \cup b)(a \cup b))^* b \cup a \cup b$ ✓
☒ $\{w \mid |w| \text{ is odd and } w \text{ starts and ends with the same symbol}\}$ ✓
☐ $a(a \cup b)^* a \cup b(a \cup b)^* b$
☐ $a(a \cup b)^* a \cup b(a \cup b)^* b \cup a \cup b$
☐ $\{w \mid w \text{ starts and ends with } a\}$
☐ $a(a \cup b)^+ a \cup b(a \cup b)^+ b \cup a \cup b$

😊 Your answer is correct.

The correct answers are: $\{w \mid |w| \text{ is odd and } w \text{ starts and ends with the same symbol}\}$,
 $a(a \cup b)((a \cup b)(a \cup b))^* a \cup b(a \cup b)((a \cup b)(a \cup b))^* b \cup a \cup b$

Question 17

Flag question

Correct

Mark 1 out of 1

Complete the context-free grammar below so that it generates the language $\{a^n b c^{2n} \mid n \geq 0\}$.

$S \rightarrow$ ☒ S ☒ ☒ $|$ ☒

 Your answer is correct.

The correct answer is:

Complete the context-free grammar below so that it generates the language $\{a^n b c^{2n} \mid n \geq 0\}$.

$S \rightarrow$ **S $|$**

Question 18

Flag question

Correct

Mark 1 out of 1

Complete the context-free grammar below so that it generates the language $\{a^n b^m \mid 0 \leq n < m\}$.

$S \rightarrow$ ☒ S ☒ $|$ ☒

$T \rightarrow bT$ $|$ ☒

 Your answer is correct.

The correct answer is:

Complete the context-free grammar below so that it generates the language $\{a^n b^m \mid 0 \leq n < m\}$.

$S \rightarrow$ **S $|$**

$T \rightarrow bT$ $|$

Question 19

Flag question

Partially correct

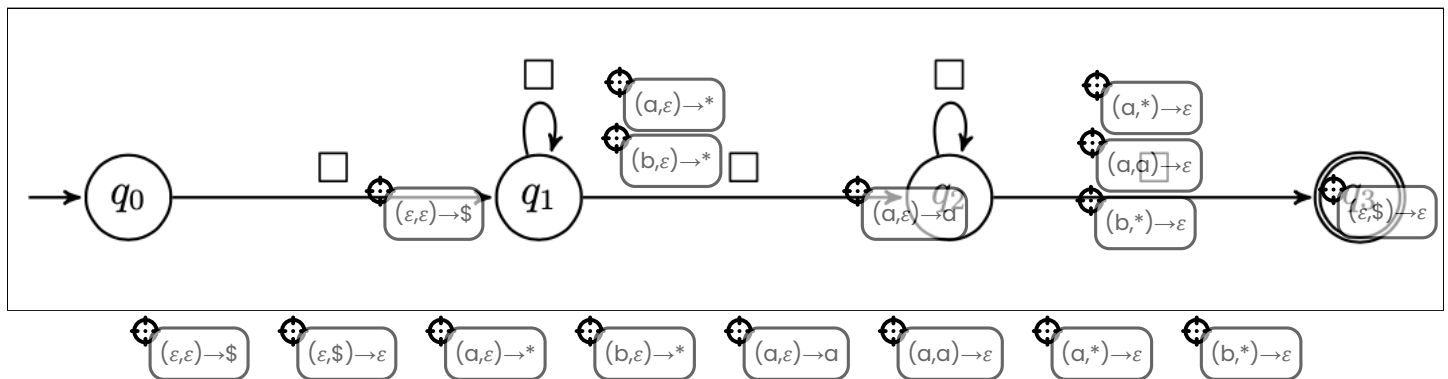
Mark 5 out of 8

Fill in the missing labels in the push-down automaton below so that it generates the language

$L = \{w \mid |w| \text{ is even and } aa \text{ is in the middle}\}.$

Instructions:

- Drag and drop the labels onto the automaton by positioning the target cursor of the label (the top left circle, not the whole label) inside the box.
- Labels can be reused several times.
- If two or more labels should go in the same box, place them on top of each other (order does not matter).



Your answer is partially correct.

You have correctly selected 5.

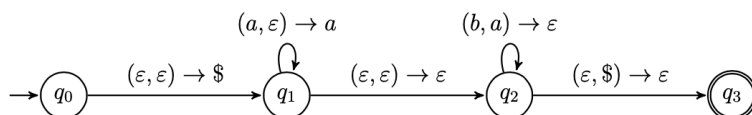
Question 20

Flag question

Correct

Mark 1 out of 1

Tick all the checkboxes corresponding to the languages recognised by the following push-down automaton



☒ $\{a^n b^n \mid n \geq 0\}$ ✓

☐ $\{w \mid w \text{ has strictly more occurrences of } a\text{'s than } b\text{'s}\}$

☐ $\{w \mid w \text{ has equal occurrences of } a\text{'s and } b\text{'s}\}$

☐ $\{a^n b^n a^n \mid n \geq 0\}$



Your answer is correct.

The correct answer is: $\{a^n b^n \mid n \geq 0\}$

Question 21

Flag question

Correct

Mark 5 out of 5

Tick all the checkboxes of the corresponding languages that are *not context-free*.

☒ $\{a^n b^m c^n \mid n, m \geq 0 \text{ and } n \leq m\}$ ✓

☒ $\{ww \mid w \in \{a, b\}^*\}$ ✓

☐ $\{a^n b c^n \mid n > 0\}$
☒ $\{w\#uw \mid w, u \in \{a, b\}^*\}$ ✓

☐ $\{ww \mid w \in \{a\}^*\}$


Your answer is correct.

The correct answers are: $\{ww \mid w \in \{a, b\}^*\}$, $\{w\#uw \mid w, u \in \{a, b\}^*\}$, $\{a^n b^m c^n \mid n, m \geq 0 \text{ and } n \leq m\}$

Question 22

Flag question

Correct

Mark 8 out of 8

Which of the following statements is true?

(Tick only the checkboxes corresponding to true statements).

☐ The complement of a context-free language is regular

☐ The intersection of two context-free languages is context-free

☒ The concatenation of two context-free languages is context-free ✓

☐ The complement of a context-free language is never context-free

☒ The complement of a context-free language might be context-free ✓

☒ The intersection of two regular languages is regular ✓

☐ The intersection of a context-free language with a regular language is regular

☒ The complement of a regular language is regular ✓


Your answer is correct.

The correct answers are:

The complement of a regular language is regular, The intersection of two regular languages is regular, The concatenation of two context-free languages is context-free, The complement of a context-free language might be context-free

Question 23

Flag question

Correct

Mark 8 out of 8

Consider the languages L_1, L_2, L_3 over $\{a, b\}$ defined below

$$L_1 = \{a^n b^m a^n \mid 0 \leq n < m\}$$

$$L_2 = \{a^n b \mid n \geq 0\}$$

$$L_3 = \{a^n b^m \mid 0 \leq n \leq m\}$$

Which of the following statements is true? (Check only the boxes corresponding to true statements)

☐ L_2 is regular but not context-free

☒ $L_2 \cap L_3$ is regular ☒ $L_2 \cap L_3 = \{ab\}$ so it is regular

☐ L_1 is context-free

☒ L_3 is context-free ☒ L_3 can be generated by the grammar $S \rightarrow aSb \mid Sb \mid \varepsilon$

☒ L_2 is context-free ☒ L_2 is equivalent to the regular expression a^*b , thus is regular. All regular languages are context-free!

☐ L_2 is context-free but not regular

☒ $L_2 \cap L_3$ is context-free ☒ L_2 is regular and L_3 is context-free. Intersection of context-free languages with regular languages is context-free.

☒ L_3 is context-free but not regular ☒ L_3 can be generated by the grammar $S \rightarrow aSb \mid Sb \mid \varepsilon$, so it is context-free. However, it is not regular as it was shown in class by using the Pumping Lemma for regular expressions.



Your answer is correct.

The correct answers are: L_2 is context-free, L_3 is context-free, L_3 is context-free but not regular, $L_2 \cap L_3$ is context-free, $L_2 \cap L_3$ is regular

Finish review



Status	Started	Completed	Duration	Grade
Finished	Sunday, 15 June 2025, 12:35 PM	Sunday, 15 June 2025, 12:38 PM	3 mins 6 secs	21.00 out of 21.00 (100%)

Question 1

Flag question

Correct

Mark 2.00 out of 2.00

Fill in the blanks in the proof tree below so that it becomes a well-formed derivation tree according to the *state model* big-step operational semantics for typed Bims statements, where

 $W = \text{while } x \leq \underline{1} \text{ do } x := x + \underline{1},$
 $s_0 = s[x \mapsto 1],$
 $s_1 = s[x \mapsto 2].$

Instructions:

- Drag and drop the items into the boxes with the same shape.
- Each item can be used zero or more times.

$[assBS]$

$s_0 \vdash x + \underline{1} \rightarrow_E 2$

$\langle x := x + \underline{1}, s_0 \rangle \rightarrow s_1$

$[compBS]$

$s_1 \vdash x \leq \underline{1} \rightarrow_E ff$

$\langle W, s_1 \rangle \rightarrow s_1$

$[while-ffBS]$

$\langle x := x + \underline{1}; W, s_0 \rangle \rightarrow s_1$

$s_0 \vdash x \leq \underline{1} \rightarrow_E tt$

$[while-ttBS]$

$\langle W, s_0 \rangle \rightarrow s_1$

$\langle x := x + \underline{1}; W, s_0 \rangle \rightarrow s_1$

$s_0 \vdash x \leq \underline{1} \rightarrow_E tt$

$\langle x := x + \underline{1}, s_0 \rangle \rightarrow s_1$

$s_0 \vdash x + \underline{1} \rightarrow_E 2$

$\langle W, s_1 \rangle \rightarrow s_1$

$s_1 \vdash x \leq \underline{1} \rightarrow_E ff$

$s_0 \vdash x \leq \underline{1} \rightarrow_E ff$

$\langle W, s_0 \rangle \rightarrow s_1$

$[while-ttBS]$

$[compBS]$

$[assBS]$

$[while-ffBS]$

$[leqBS]$

$[sumBS]$

Your answer is correct.

Question 2

Flag question

Correct

Mark 2.00 out of 2.00

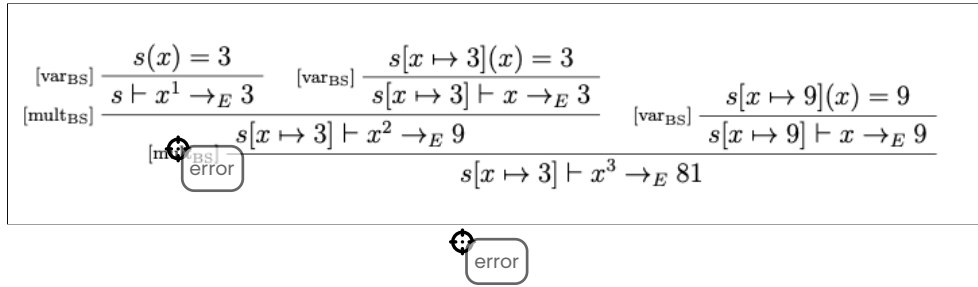
Consider the following Bims expression, conductively defined on $n \in \mathbb{N}$.

$$e^0 = \underline{1},$$

$$e^{n+1} = e^n \cdot e.$$

Below there is a **wrong** proposal of a derivation tree using the big-step operational semantics for typed Bims expressions for the evaluation of x^3 , where we assume $s = [x \mapsto 3]$.

Locate all the mistakes by dragging the "error marker" on top of the labels of the derivation rules that have been wrongly applied.



😊 Your answer is correct.

The last rule $[mult_{BS}]$ (the bottom one) is wrongly applied because the premises use two different states, but should be the same state.

Question 3

Flag question

Correct

Mark 5.00 out of 5.00

Let $W = \text{while } x \leq 9 \text{ do } x := x + 3$. Fill in the blanks in the derivation sequence below according to the rules of the state model small-step semantics for typed Bims statements. (Expand the window of your browsers for better visualisation).

$\langle \text{var Int } x := 0; W, \emptyset \rangle$
 $\Rightarrow \langle W, [x \mapsto 0] \rangle$ (dec_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 0] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 3] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 3] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 3] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 6] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 6] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 9] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 9] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 9] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 12] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 12] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 12] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 12]$ (skip_{SS})

 Solution:

$\langle \text{var Int } x := 0; W, \emptyset \rangle$
 $\Rightarrow \langle W, [x \mapsto 0] \rangle$ (dec_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 0] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 0] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 3] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 3] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 3] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 6] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 6] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 6] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 9] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 9] \rangle$ (while_{SS})
 $\Rightarrow \langle x := x + 3; W, [x \mapsto 9] \rangle$ (if-tt_{SS})
 $\Rightarrow \langle W, [x \mapsto 12] \rangle$ (comp-2_{SS})
 $\Rightarrow \langle \text{if } x \leq 9 \text{ then } x := x + 3; W \text{ else skip}, [x \mapsto 12] \rangle$ (while_{SS})
 $\Rightarrow \langle \text{skip}, [x \mapsto 12] \rangle$ (if-ff_{SS})
 $\Rightarrow [x \mapsto 12]$ (skip_{SS})

Question 4

Flag question

Correct

Mark 2.00 out of 2.00

Consider the typed Bip statement

```
S = proc p is x := y + 3;
  begin
    var lnt y := 10;
    call p
  end
```

Fill in the blanks in the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip statements using **fully-dynamic scope rules**.

$$\begin{array}{c}
 \sigma_1 \circ \boxed{\mathcal{E}_1} \vdash y + \underline{3} \rightarrow_E \boxed{13} \\
 \text{[ass}_{BS}\text{]} \frac{}{\boxed{\mathcal{E}_1} \pi_0 \vdash_{l_3} \langle \boxed{x := y + 3} \sigma_1 \rangle \rightarrow \sigma_2 \quad \pi_0(p) = \boxed{x := y + 3}} \\
 \text{[call}_{BS}\text{]} \frac{}{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{10} \rightarrow_E 10 \quad \mathcal{E}_1, \pi_0 \vdash_{l_3} \langle \text{call } p, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \text{[dec}_{BS}\text{]} \frac{}{\mathcal{E}_0, \pi_0 \vdash_{l_2} \langle \text{var lnt } y := \underline{10}; \text{call } p, \sigma_0 \rangle \rightarrow \sigma_2} \\
 \text{[block}_{BS}\text{]} \frac{}{\mathcal{E}_0, \pi_0 \vdash_{l_2} \langle \text{begin var lnt } y := \underline{10}; \text{call } p \text{ end}, \sigma_0 \rangle \rightarrow \sigma_2} \\
 \text{[proc}_{BS}\text{]} \frac{}{\mathcal{E}_0, \emptyset \vdash_{l_2} \langle S, \sigma_0 \rangle \rightarrow \sigma_2}
 \end{array}$$

where

$$\begin{array}{ll}
 \sigma_0 = \sigma[l_0 \mapsto 0, l_1 \mapsto 0] & \mathcal{E}_0 = \mathcal{E}[x \mapsto l_0, y \mapsto l_1] \\
 \sigma_1 = \sigma[l_0 \mapsto 0, l_1 \mapsto 0, l_2 \mapsto 10] & \mathcal{E}_1 = \mathcal{E}[x \mapsto l_0, y \mapsto l_2] \\
 \sigma_2 = \sigma[l_0 \mapsto \boxed{13}, l_1 \mapsto 0, l_2 \mapsto 10] & \pi_0 = [p \mapsto \boxed{x := y + 3}]
 \end{array}$$

$$\begin{array}{c}
 (\mathcal{E}_0, x := y + \underline{3}) \\
 \boxed{3} \\
 \boxed{\mathcal{E}_0}
 \end{array}$$

😊 Your answer is correct.

Question 5

Flag question

Correct

Mark 2.00 out of 2.00

Consider the typed Bip statement

```
S = proc p is x := y + 3;
  begin
    var Int y := 10;
    call p
  end
```

Fill in the blanks in the following derivation tree constructed according to the rules of the *environment-store model* big-step operational semantics for typed Bip statements using mixed scope rules with **static variable bindings and dynamic procedure names bindings**.

$$\begin{array}{c}
 \sigma_1 \circ \boxed{\mathcal{E}_0} \vdash y + \underline{3} \rightarrow_E \boxed{3} \\
 \text{[ass}_{\text{BS}}] \frac{}{\boxed{\mathcal{E}_0} \pi_0 \vdash_{l_3} \langle x := y + \underline{3} \rangle \sigma_1 \rightarrow \sigma_2 \quad \pi_0(p) = (\mathcal{E}_0, x := y + \underline{3})} \\
 \text{[call}_{\text{BS}}] \frac{}{\sigma_0 \circ \mathcal{E}_0 \vdash \underline{10} \rightarrow_E 10 \quad \mathcal{E}_1, \pi_0 \vdash_{l_3} \langle \text{call } p, \sigma_1 \rangle \rightarrow \sigma_2} \\
 \text{[dec}_{\text{BS}}] \frac{}{\mathcal{E}_0, \pi_0 \vdash_{l_2} \langle \text{var Int } y := \underline{10}; \text{call } p, \sigma_0 \rangle \rightarrow \sigma_2} \\
 \text{[block}_{\text{BS}}] \frac{}{\mathcal{E}_0, \pi_0 \vdash_{l_2} \langle \text{begin var Int } y := \underline{10}; \text{call } p \text{ end}, \sigma_0 \rangle \rightarrow \sigma_2} \\
 \text{[proc}_{\text{BS}}] \frac{}{\mathcal{E}_0, \emptyset \vdash_{l_2} \langle S, \sigma_0 \rangle \rightarrow \sigma_2}
 \end{array}$$

where

$$\begin{array}{ll}
 \sigma_0 = \sigma[l_0 \mapsto 0, l_1 \mapsto 0] & \mathcal{E}_0 = \mathcal{E}[x \mapsto l_0, y \mapsto l_1] \\
 \sigma_1 = \sigma[l_0 \mapsto 0, l_1 \mapsto 0, l_2 \mapsto 10] & \mathcal{E}_1 = \mathcal{E}[x \mapsto l_0, y \mapsto l_2] \\
 \sigma_2 = \sigma[l_0 \mapsto \boxed{3}, l_1 \mapsto 0, l_2 \mapsto 10] & \pi_0 = [p \mapsto (\mathcal{E}_0, x := y + \underline{3})]
 \end{array}$$

$$\boxed{x := y + \underline{3}} \quad \boxed{(\mathcal{E}_0, x := y + \underline{3})}$$

$$\boxed{13}$$

$$\boxed{\mathcal{E}_1}$$

😊 Your answer is correct.

Question 6

[Flag question](#)

Correct

Mark 8.00 out of 8.00

Consider the typed Bip statement below:

```
var Int x := 10;  
var Int y := -2;  
proc p is y := x + 1;  
proc q is call p;  
begin  
  var Int x := 3;  
  proc p is x := x + 2;  
  call q;  
  y := x  
end
```

What is the final value assigned to the (global) variable y under the following scope rules?

- Fully-static scope rule (static variable bindings and static procedure name bindings): $y =$ ✓
- Mixed scope rule (dynamic variable bindings and static procedure name bindings): $y =$ ✓
- Mixed scope rule (static variable bindings and dynamic procedure name bindings): $y =$ ✓
- Fully-dynamic scope rule (dynamic variable bindings and dynamic procedure name bindings): $y =$ ✓

😊 Full details regarding the derivations are [here](#) (click the link to download the pdf).

[Finish review](#)